

ИУ7 «ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ЭВМ И ИНФОРМАЦИОННЫЕ ТЕХНОЛОГИИ»

Разработка базы данных для системы онлайн-заказа еды

(И.О. Фамилия)

2026 г.

РЕФЕРАТ

Расчётно-пояснительная записка содержит 69 страниц, 13 рисунков, 22 таблицы, 28 источников.

Ключевые слова: база данных, реляционная база данных, PostgreSQL, Redis, кэширование, индексация, REST API, онлайн-заказ еды, триггер, хранимая функция, ролевая модель.

Цель работы — проектирование и реализация базы данных для системы онлайн-заказа еды, а также приложения для доступа к ней.

В аналитическом разделе проведён анализ предметной области и существующих систем онлайн-заказа еды, сформулированы требования к разрабатываемой базе данных и приложению, выделены роли пользователей системы.

В конструкторском разделе спроектирована база данных, описаны ограничения целостности, разработаны алгоритмы хранимой функции расчёта итоговой стоимости заказа и триггеров обновления рейтингов, спроектирована ролевая модель на уровне базы данных.

В технологическом разделе обоснован выбор средств реализации, реализованы база данных и приложение, описан интерфейс доступа к базе данных.

В исследовательском разделе проведено исследование влияния внешнего кэширования и индексации на время выполнения запросов к базе данных.

СОДЕРЖАНИЕ

РЕФЕРАТ	4
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ	7
ВВЕДЕНИЕ	8
1 Аналитический раздел	9
1.1 Анализ предметной области	9
1.2 Сравнительный анализ существующих решений	9
1.3 Требования к базе данных и приложению	11
1.4 Формализация данных и связей между ними	13
1.5 Анализ моделей данных	15
1.6 Формализация и описание пользователей	16
1.7 Вывод	17
2 Конструкторский раздел	18
2.1 Проектирование базы данных	18
2.2 Проектирование функции и триггеров	28
2.3 Проектирование ролевой модели	31
2.4 Проектирование приложения	33
2.5 Вывод	35
3 Технологический раздел	36
3.1 Выбор средств реализации базы данных	36
3.2 Выбор In-Memory СУБД	37
3.3 Выбор средств реализации приложения	37
3.4 Создание таблиц и типов базы данных	37
3.5 Реализация ограничений целостности	43
3.6 Создание функции и триггеров	48
3.7 Тестирование разработанных функций	49
3.8 Создание ролевой модели	50
3.9 Реализация взаимодействия PostgreSQL с Redis	51
3.10 Описание интерфейса доступа к базе данных	52

3.11 Вывод	54
4 Исследовательский раздел	56
4.1 Технические характеристики ЭВМ	56
4.2 Описание исследования	56
4.2.1 Исследование влияния кэширования	56
4.2.2 Исследование влияния индексов	57
4.3 Результаты исследования	57
4.3.1 Влияние кэширования	57
4.3.2 Влияние индексов	59
4.4 Вывод	60
ЗАКЛЮЧЕНИЕ	61
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	64
ПРИЛОЖЕНИЕ А	65
ПРИЛОЖЕНИЕ Б	66

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей расчётно-пояснительной записке применяют следующие сокращения и обозначения.

API — Application Programming Interface (интерфейс программирования приложений)

ACID — Atomicity, Consistency, Isolation, Durability (атомарность, согласованность, изолированность, долговечность)

Cache hit — попадание в кэш — ситуация, при которой запрашиваемые данные найдены в кэше

Cache miss — промах кэша — ситуация, при которой запрашиваемые данные отсутствуют в кэше

JWT — JSON Web Token (веб-токен на основе JSON) — стандарт передачи данных аутентификации

REST — Representational State Transfer (передача репрезентативного состояния) — архитектурный стиль взаимодействия компонентов распределённого приложения

SQL — Structured Query Language (язык структурированных запросов)

TTL — Time To Live (время жизни) — период времени, по истечении которого данные в кэше считаются устаревшими и удаляются

БД — база данных

СУБД — система управления базами данных

ВВЕДЕНИЕ

Современные системы онлайн-доставки еды ежедневно обрабатывают большое количество заказов, координируют работу тысяч ресторанов и курьеров, управляют платежами и отслеживают статус доставок. Основной причиной развития таких систем является повседневное использование мобильных приложений, а также пандемия COVID-19, закрепившая у пользователей привычку к бесконтактным услугам [1].

Согласно актуальным статистическим данным, российский рынок готовой еды в 2025 году демонстрирует устойчивую тенденцию к росту. Ключевым фактором развития отрасли является не только рост спроса со стороны потребителей, но и значительное увеличение выручки производителей. При этом рынок сталкивается с рядом вызовов: ростом себестоимости, дефицитом производственных мощностей и необходимостью цифровизации для оптимизации логистических процессов [2].

В основе систем онлайн-доставки еды лежит база данных, которая должна обеспечивать быстрый доступ к информации, гарантировать целостность финансовых транзакций и хранить историю операций. От качества проектирования базы данных зависит скорость обработки заказов, точность расчётов и возможность масштабирования системы.

Цель работы: разработка базы данных для системы онлайн-заказов еды, а также приложения для доступа к ней.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ существующих систем онлайн-заказов еды;
- определить требования и ограничения к разрабатываемой базе данных и приложению;
- спроектировать базу данных и задать ограничения целостности;
- выбрать средства реализации базы данных и приложения;
- реализовать спроектированные базу данных и приложение;
- провести исследование влияния внешнего кэширования и индексации на время выполнения запросов к базе данных.

1 Аналитический раздел

1.1 Анализ предметной области

Система онлайн-заказа еды представляет собой агрегатор, связывающий клиентов, рестораны и курьеров в единой платформе. Клиент выбирает ресторан, формирует заказ из доступных блюд, указывает адрес доставки и способ оплаты. Ресторан подтверждает заказ и приступает к приготовлению, после чего передаёт его курьеру. Курьер осуществляет доставку и обновляет статус заказа на каждом этапе, что позволяет клиенту отслеживать его в режиме реального времени.

Меню ресторана организуется по категориям блюд. При этом одно блюдо может относиться к нескольким категориям одновременно. Клиенты фильтруют блюда по категории, наличию и диапазону цен [3].

Стоимость доставки формируется динамически и зависит от удалённости адреса клиента от ресторана, времени суток и дня недели. В часы пик и в выходные дни стоимость возрастает ввиду повышенного спроса. Каждый ресторан может самостоятельно устанавливать зоны доставки с собственными тарифами и минимальную сумму заказа, при достижении которой доставка предоставляется бесплатно [4].

После завершения доставки клиент может оставить отзыв с оценкой ресторана, курьера и скорости доставки. На основе накопленных оценок автоматически формируется рейтинг ресторана и курьера [5].

1.2 Сравнительный анализ существующих решений

В рамках анализа существующих решений были выбраны следующие сервисы: «Яндекс Еда» [6], «Деливери» [7], «Купер» [8]. «Яндекс Еда» и «Купер» также предоставляют доставку продуктов из магазинов, однако в рамках данного анализа рассматривается их функциональность в части доставки готовых блюд из ресторанов.

Для сравнения существующих решений были выделены критерии, отражающие как базовые функции подобных сервисов, так и специфические

правила организации доставки и обработки заказов:

- 1) каталог блюд с категориями;
- 2) несколько адресов доставки у клиента с выбором основного;
- 3) отслеживание статуса заказа;
- 4) история заказов;
- 5) отзывы и рейтинги;
- 6) поддержка нескольких способов оплаты;
- 7) динамическое изменение стоимости доставки в зависимости от времени суток, дня недели и удалённости адреса клиента;
- 8) возможность заведения устанавливая собственные условия доставки, включая минимальную сумму заказа, зоны обслуживания и временные интервалы работы;
- 9) наличие открытого публичного API для интеграции с внешними программами

В таблице 1.1 приводится сравнение существующих решений по выделенным критериям.

Таблица 1.1 – Сравнительный анализ существующих систем онлайн-заказа еды

Критерий	Система онлайн-заказа еды		
	Яндекс Еда	Деливери	Купер
Каталог блюд с категориями	Да	Да	Да
Несколько адресов у клиента	Да	Да	Да
Отслеживание статуса заказа	Да	Да	Да
История заказов	Да	Да	Да
Отзывы и рейтинги	Да	Да	Да
Несколько способов оплаты	Да	Да	Да
Динамическое изменение стоимости доставки	Да	Да	Да
Собственные условия доставки у заведения	Да	Да	Да
Открытый публичный API	Нет	Нет	Нет

По результатам анализа установлено, что все рассмотренные системы реализуют единый базовый набор функций: ведение каталога блюд с категориями, хранение нескольких адресов клиента, отслеживание статуса заказа, историю заказов, отзывы и рейтинги, несколько способов оплаты, динамическое изменение стоимости доставки, а также наличие у ресторанов собственных условий доставки. Вместе с тем ни одна из рассмотренных систем не предоставляет открытого публичного API, что открывало бы возможность для сторонних разработчиков интегрировать функциональность системы в собственные приложения. На основании этого определён перечень ключевых процессов предметной области:

- представление каталога блюд заведений с фото, описаниями и ценами;
- управление профилем пользователя и хранение нескольких адресов доставки;
- интеграция платёжных систем;
- отслеживание статуса заказа в режиме реального времени;
- хранение истории заказов и отзывов клиентов;
- динамическое формирование стоимости доставки в зависимости от различных факторов;
- предоставление заведениям возможности устанавливать собственные условия доставки;
- предоставление открытого публичного API.

1.3 Требования к базе данных и приложению

К функциональным требованиям разрабатываемой базы данных относятся:

- хранение данных о клиентах;
- хранение данных о ресторанах;
- хранение данных о курьерах;
- регистрация и хранение заказов;
- обеспечение связи между заказом, клиентом, рестораном и курьером;
- хранение отзывов и оценок клиентов;
- хранение истории заказов для каждого клиента.

К нефункциональным требованиям разрабатываемой базы данных относятся:

- целостность данных: использование ограничений внешних ключей и запрет на хранение неопределённых значений в обязательных полях;
- нормализация схемы: структура базы данных должна соответствовать третьей нормальной форме;
- масштабируемость: схема должна допускать добавление новых сущностей без перестройки существующей структуры.

К функциональным требованиям разрабатываемого приложения относятся:

- регистрация и аутентификация пользователей;
- просмотр меню ресторанов с фильтрацией по категориям и цене;
- оформление заказа с выбором блюд, адреса доставки и способа оплаты;
- просмотр истории заказов клиентом;
- возможность оставлять отзывы и выставлять оценки ресторанам после завершения заказа;
- управление данными ресторанов, меню и курьеров;
- предоставление открытого публичного API для интеграции с внешними программами.

К нефункциональным требованиям разрабатываемого приложения относятся:

- безопасность: хранение паролей в хэшированном виде, аутентификация на основе токенов;
- надёжность: корректная обработка ошибок и возврат информативных сообщений при некорректных запросах;
- масштабируемость: архитектура должна допускать подключение новых модулей без изменения существующих компонентов;
- документированность: все точки доступа к API должны быть описаны в соответствии с единым стандартом и доступны через интерактивную документацию.

1.4 Формализация данных и связей между ними

Разрабатываемая база данных должна хранить информацию о следующих сущностях:

- клиент — ФИО, контактная информация, дата регистрации, пароль;
- адреса клиента — адрес доставки, привязка к клиенту;
- курьер — ФИО, контактная информация, статус доступности;
- ресторан — наименование, адрес, контактная информация;
- администратор — электронная почта, роль (администратор системы, администратор ресторана), привязка к ресторану;
- зоны доставки — географические границы, сумма доставки, привязка к ресторану;
- заказ — статус, дата оформления, привязка к клиенту, курьеру и ресторану;
- детали заказа — состав заказа, количество и стоимость позиций;
- категории блюд — наименование категории, привязка к ресторану;
- блюда — наименование, описание, цена, привязка к категории;
- отзывы — оценка, текст, дата публикации, привязка к клиенту и ресторану;
- платежи — сумма, способ оплаты, статус, привязка к заказу.

На рисунке 1.1 представлена диаграмма сущность-связь модели предметной области в нотации Чена.

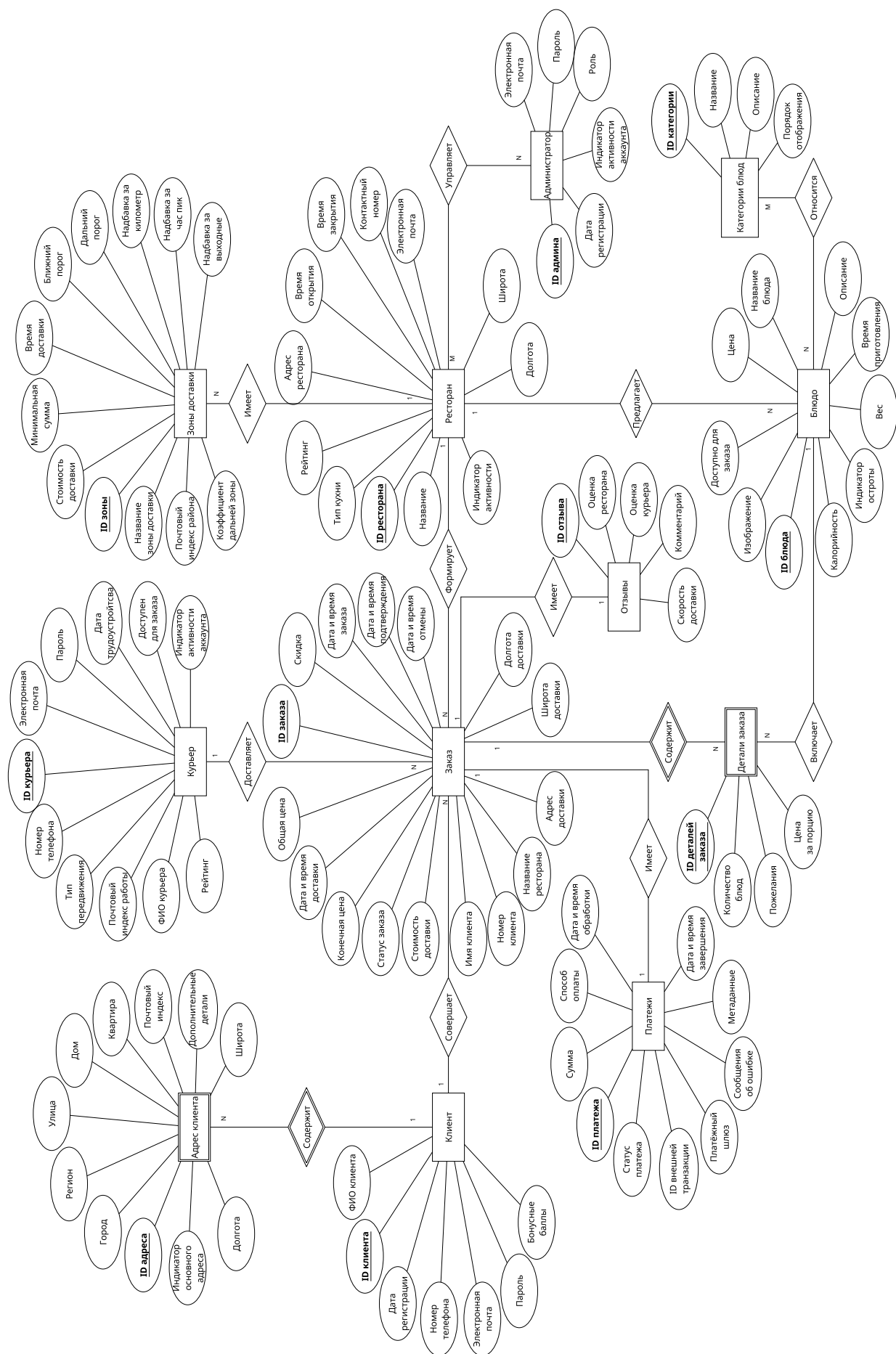


Рисунок 1.1 – Диаграмма сущность-связь

1.5 Анализ моделей данных

Дореляционные модели представлены инвертированными списками, иерархической и сетевой моделями. Иерархическая модель организует данные в виде дерева, где каждый дочерний узел имеет ровно одного родителя. Сетевая модель расширяет иерархическую, допуская наличие нескольких родительских узлов. Инвертированные списки представляют данные в виде таблиц, но для поисковых атрибутов создаются отдельные индексы, которые содержат упорядоченные списки значений с указателями на соответствующие записи основной таблицы (инвертированный список). Это обеспечивает быстрый поиск по нескольким условиям одновременно. Иерархическая и сетевая модели обеспечивают высокую производительность при заранее известных маршрутах доступа к данным, тогда как инвертированные списки ускоряют поиск по любому индексированному атрибуту. Однако все три модели обладают жёсткой структурой и требуют значительных затрат при модификации схемы [9].

Реляционная модель, предложенная Э. Коддом в 1970 году, основывается на математической теории отношений и представляет данные исключительно в виде таблиц [9]. Связи в этой модели устанавливаются через явное задание значений атрибутов, таких как внешние ключи. Модель обеспечивает независимость данных от способов их физического хранения, поддерживает декларативный язык запросов SQL и гарантирует выполнение требований ACID через механизм транзакций. Реляционные СУБД получили доминирующее распространение благодаря своей гибкости и строгому математическому аппарату [10].

Постреляционные модели включают документные, графовые, столбцовые хранилища и системы типа «ключ-значение». Они разрабатывались для преодоления ограничений реляционного подхода при работе со слабоструктурированными данными, необходимости горизонтального масштабирования и экстремальных нагрузках в распределенных средах. Большинство таких решений (класса NoSQL) обеспечивают высокую производительность и доступность за счет ослабления требований к транзакционности и строгой согласованности данных [11].

Для разрабатываемой базы данных выбрана реляционная модель, по-

скольку предметная область характеризуется чётко определёнными связями между сущностями, необходимостью обеспечения целостности данных и поддержкой сложных запросов с агрегацией.

1.6 Формализация и описание пользователей

Взаимодействовать с разрабатываемой системой онлайн-заказов еды будут следующие роли пользователей:

- гость;
- клиент;
- курьер;
- администратор ресторана;
- администратор системы.

Гость — неавторизованный пользователь, может только просматривать рестораны и меню, но не может совершать заказы.

Клиент — основная роль системы, может выбрать адрес доставки, просматривать доступные рестораны и их меню, совершать заказы, просматривать историю заказов, отслеживать их статус и оставлять отзыв о заказе.

Курьер принимает назначенные заказы и управляет статусом их доставки.

Администратор ресторана управляет меню и заказами своего ресторана.

Администратор системы управляет всей платформой.

На рисунке 1.2 представлена диаграмма вариантов использования разрабатываемой системы.



Рисунок 1.2 – Диаграмма вариантов использования

1.7 Вывод

В аналитическом разделе был проведён анализ предметной области, сформулированы требования и ограничения к разрабатываемой базе данных и приложению, выделены сущности предметной области и описаны связи между ними, был проведён анализ моделей данных и была выбрана реляционная модель на основе формализации данных, выделены роли и описание пользователей системы.

2 Конструкторский раздел

2.1 Проектирование базы данных

В соответствии с формализацией данных, описанной в аналитической части, и рисунком 1.1 были выделены следующие таблицы базы данных:

- **customers** — таблица клиентов;
- **customer_addresses** — таблица адресов клиентов;
- **couriers** — таблица курьеров;
- **restaurants** — таблица ресторанов;
- **admins** — таблица администраторов;
- **admin_restaurants** — таблица связи администраторов и ресторанов;
- **delivery_zones** — таблица зон доставки;
- **orders** — таблица заказов;
- **order_items** — таблица деталей заказов;
- **dish_category_mapping** — таблица связи блюд и категорий;
- **dishes** — таблица блюд;
- **dish_categories** — таблица категории блюд;
- **payments** — таблица платежей;
- **reviews** — таблица отзывов.

В таблицах 2.1–2.14 представлена информация о спроектированных таблицах, а также описаны ограничения целостности данных.

Таблица 2.1 – Информация о таблице клиентов

Атрибут	Значение	Тип данных	Ограничение
customer_id	Идентификатор	UUID	Первичный ключ, не нулевой
full_name	Полное имя покупателя	Строка	Не нулевой
phone	Номер телефона	Строка	Не нулевой, уникальный
email	Электронная почта	Строка	Не нулевой, уникальный
password_hash	Хэш пароля	Строка	Не нулевой
bonuses	Количество бонусов	Целое число	Неотрицательное число, по умолчанию 0
is_active	Индикатор активности аккаунта	Логический	По умолчанию «Истина»
created_at	Дата регистрации	Дата и время	По умолчанию «Текущая дата»
updated_at	Дата изменения	Дата и время	По умолчанию «Текущая дата»

Таблица 2.2 – Информация о таблице адресов клиента

Атрибут	Значение	Тип данных	Ограничение
address_id	Идентификатор	UUID	Первичный ключ, не нулевой
customer_id	Идентификатор	UUID	Внешний ключ, не нулевой
region	Регион	Строка	
city	Город	Строка	Не нулевой
street	Улица	Строка	Не нулевой
house	Номер дома	Строка	Не нулевой
apartment	Квартира	Строка	
address_details	Дополнительные детали адреса	Строка	
postal_code	Почтовый индекс	Строка	
latitude	Широта	Вещественное число	
longitude	Долгота	Вещественное число	
is_default	Индикатор основного адреса	Логический	По умолчанию «Ложь», допускается не более одной истинной записи на клиента
created_at	Дата создания	Дата и время	По умолчанию «Текущая дата»

Таблица 2.3 – Информация о таблице курьеров

Атрибут	Значение	Тип данных	Ограничение
courier_id	Идентификатор	UUID	Первичный ключ, не нулевой
full_name	Полное имя курьера	Строка	Не нулевой
phone	Номер телефона	Строка	Не нулевой, уникальный
email	Электронная почта	Строка	Не нулевой, уникальный
password_hash	Хэш пароля	Строка	Не нулевой
employee_date	Дата трудоустройства	Дата	Не нулевой
area_of_work	Область работы	Строка	
vehicle_type	Тип передвижения	Перечисление	
rating	Рейтинг курьера	Вещественное число	Неотрицательное, не больше 5
is_available	Индикатор готовности к заказам	Логический	По умолчанию «Истина»
is_active	Индикатор активности аккаунта	Логический	По умолчанию «Истина»
created_at	Дата регистрации	Дата и время	По умолчанию «Текущая дата»
updated_at	Дата изменения	Дата и время	По умолчанию «Текущая дата»

Таблица 2.4 – Информация о таблице администраторов

Атрибут	Значение	Тип данных	Ограничение
admin_id	Идентификатор	UUID	Первичный ключ, не нулевой
email	Электронная почта	Строка	Не нулевой, уникальный
password_hash	Хэш пароля	Строка	Не нулевой
role	Роль администратора	Перечисление	Не нулевой
is_active	Индикатор активности аккаунта	Логический	По умолчанию «Истина»
created_at	Дата регистрации	Дата и время	По умолчанию «Текущая дата»
updated_at	Дата изменения	Дата и время	По умолчанию «Текущая дата»

Таблица 2.5 – Информация о таблице ресторанов

Атрибут	Значение	Тип данных	Ограничение
restaurant_id	Идентификатор	UUID	Первичный ключ, не нулевой
name	Название ресторана	Строка	Не нулевой
cuisine_type	Тип кухни	Строка	
rating	Рейтинг ресторана	Вещественное число	Неотрицательное, не больше 5
address	Адрес	Строка	Не нулевой
phone	Номер телефона	Строка	Не нулевой, уникальный
email	Электронная почта	Строка	Не нулевой, уникальный
latitude	Широта	Вещественное число	
longitude	Долгота	Вещественное число	
opening_time	Время открытия	Время	
closing_time	Время закрытия	Время	
is_active	Индикатор активности ресторана	Логический	По умолчанию «Истина»
created_at	Дата регистрации	Дата и время	По умолчанию «Текущая дата»
updated_at	Дата изменения	Дата и время	По умолчанию «Текущая дата»

Таблица 2.6 – Информация о таблице связи администраторов и ресторанов

Атрибут	Значение	Тип данных	Ограничение
admin_id	Идентификатор администратора	UUID	Внешний ключ, не нулевой
restaurant_id	Идентификатор ресторана	UUID	Внешний ключ, не нулевой

Таблица 2.7 – Информация о таблице зон доставки

Атрибут	Значение	Тип данных	Ограничение
zone_id	Идентификатор	UUID	Первичный ключ, не нулевой
restaurant_id	Идентификатор ресторана	UUID	Внешний ключ, не нулевой
zone_name	Название зоны доставки	Строка	Не нулевой
postal_code	Почтовый индекс района	Строка	Не нулевой
delivery_fee	Базовая стоимость доставки	Вещественное число	Неотрицательное, по умолчанию 0
near_threshold	Ближний порог расстояния (км)	Вещественное число	Положительное, по умолчанию 3, $\text{near_threshold} \leq \text{far_threshold}$
far_threshold	Дальний порог расстояния (км)	Вещественное число	Положительное, по умолчанию 5, $\text{far_threshold} \geq \text{near_threshold}$
fee_per_km	Надбавка за километр сверх ближнего порога	Вещественное число	Неотрицательное, по умолчанию 0
far_zone_multiplier	Коэффициент дальней зоны	Вещественное число	Положительное, неотрицательное, по умолчанию 1.5
peak_surcharge	Надбавка за доставку в час пик	Вещественное число	Неотрицательное, по умолчанию 0
weekend_surcharge	Надбавка за доставку в выходные дни	Вещественное число	Неотрицательное, по умолчанию 0
min_order_amount	Минимальная сумма заказа для бесплатной доставки	Вещественное число	Неотрицательное, по умолчанию 0
delivery_time	Время доставки в минутах	Целое число	Неотрицательное

Таблица 2.8 – Информация о таблице категории блюд

Атрибут	Значение	Тип данных	Ограничение
category_id	Идентификатор	UUID	Первичный ключ, не нулевой
name	Название категории	Строка	Не нулевой
description	Описание	Строка	
display_order	Порядок отображения	Целое число	По умолчанию 0
created_at	Дата добавления	Дата и время	По умолчанию «Текущая дата»

Таблица 2.9 – Информация о таблице блюд

Атрибут	Значение	Тип данных	Ограничение
dish_id	Идентификатор	UUID	Первичный ключ, не нулевой
restaurant_id	Идентификатор	UUID	Внешний ключ, не нулевой
name	Название блюда	Строка	Не нулевой
description	Описание	Строка	
price	Цена	Вещественное	Положительное
image_url	Путь к изображению	Строка	
is_available	Индикатор доступности	Логический	По умолчанию «Истина»
is_spicy	Индикатор остроты	Логический	По умолчанию «Ложь»
preparation_time	Время приготовления	Целое число	Неотрицательное
calories	Калорийность	Целое число	Неотрицательное
weight_grams	Вес	Целое число	Неотрицательное
created_at	Дата добавления	Дата и время	По умолчанию «Текущая дата»
updated_at	Дата изменения	Дата и время	По умолчанию «Текущая дата»

Таблица 2.10 – Информация о таблице связи блюд и категорий

Атрибут	Значение	Тип данных	Ограничение
dish_id	Идентификатор блюда	UUID	Внешний ключ, не нулевой
category_id	Идентификатор категории	UUID	Внешний ключ, не нулевой

Таблица 2.11 – Информация о таблице заказов

Атрибут	Значение	Тип данных	Ограничение
order_id	Идентификатор	UUID	Первичный ключ, не нулевой
order_number	Номер заказа	Строка	Не нулевой
customer_id	Идентификатор	UUID	Внешний ключ, не нулевой
restaurant_id	Идентификатор	UUID	Внешний ключ, не нулевой
courier_id	Идентификатор	UUID	Внешний ключ, не нулевой
customer_name	Имя клиента	Строка	Не нулевой
customer_phone	Номер клиента	Строка	Не нулевой
restaurant_name	Название ресторана	Строка	Не нулевой
delivery_address	Адрес доставки	Строка	Не нулевой
delivery_latitude	Широта доставки	Вещественное число	Не нулевой
delivery_longitude	Долгота доставки	Вещественное число	Не нулевой
subtotal	Общая цена	Вещественное	Неотрицательное
delivery_fee	Стоимость доставки	Вещественное	Неотрицательное, по умолчанию 0
discount	Скидка	Вещественное	Неотрицательное, по умолчанию 0
total_amount	Конечная цена	Вещественное	Неотрицательное
status	Статус заказа	Перечисление	По умолчанию «В ожидании»
created_at	Дата и время создания	Дата и время	По умолчанию «Текущая дата»
confirmed_at	Дата и время подтверждения заказа	Дата и время	По умолчанию «Текущая дата»
delivered_at	Дата и время доставки	Дата и время	
cancelled_at	Дата и время отмены	Дата и время	
updated_at	Дата изменения	Дата и время	По умолчанию «Текущая дата»

Таблица 2.12 – Информация о таблице деталей заказов

Атрибут	Значение	Тип данных	Ограничение
order_item_id	Идентификатор	UUID	Первичный ключ, не нулевой
order_id	Идентификатор	UUID	Внешний ключ, не нулевой
dish_id	Идентификатор	UUID	Внешний ключ, не нулевой
unit_price	Цена за одно блюдо	Вещественное число	Неотрицательное
quantity	Количество блюд	Целое число	Неотрицательное
special_requests	Особые пожелания	Строка	
created_at	Дата и время создания	Дата и время	По умолчанию «Текущая дата»

Таблица 2.13 – Информация о таблице отзывов

Атрибут	Значение	Тип данных	Ограничение
review_id	Идентификатор	UUID	Первичный ключ, не нулевой
order_id	Идентификатор	UUID	Внешний ключ, не нулевой
restaurant_rating	Оценка ресторана	Целое число	От 1 до 5
courier_rating	Оценка курьера	Целое число	От 1 до 5
delivery_speed	Оценка скорости доставки	Целое число	От 1 до 5
comment	Комментарий	Строка	
created_at	Дата и время создания	Дата и время	По умолчанию «Текущая дата»
updated_at	Дата и время изменения	Дата и время	По умолчанию «Текущая дата»

Таблица 2.14 – Информация о таблице платежей

Атрибут	Значение	Тип данных	Ограничение
payment_id	Идентификатор	UUID	Первичный ключ, не нулевой
order_id	Идентификатор	UUID	Внешний ключ, не нулевой, уникальный
amount	Сумма	Вещественное число	Положительное
payment_method	Способ оплаты	Перечисление	Не нулевой
status	Статус оплаты	Перечисление	Не нулевой, по умолчанию «В ожидании»
external_transaction_id	Идентификатор внешней транзакции	Строка	
payment_gateway	Платёжный шлюз	Строка	
error_message	Сообщение об ошибке	Строка	
metadata	Метаданные	JSON	
created_at	Дата и время создания	Дата и время	По умолчанию «Текущая дата»
processed_at	Дата и время обработки	Дата и время	
completed_at	Дата и время завершения	Дата и время	По умолчанию «Текущая дата»

На рисунке 2.1 приведена диаграмма спроектированной базы данных.

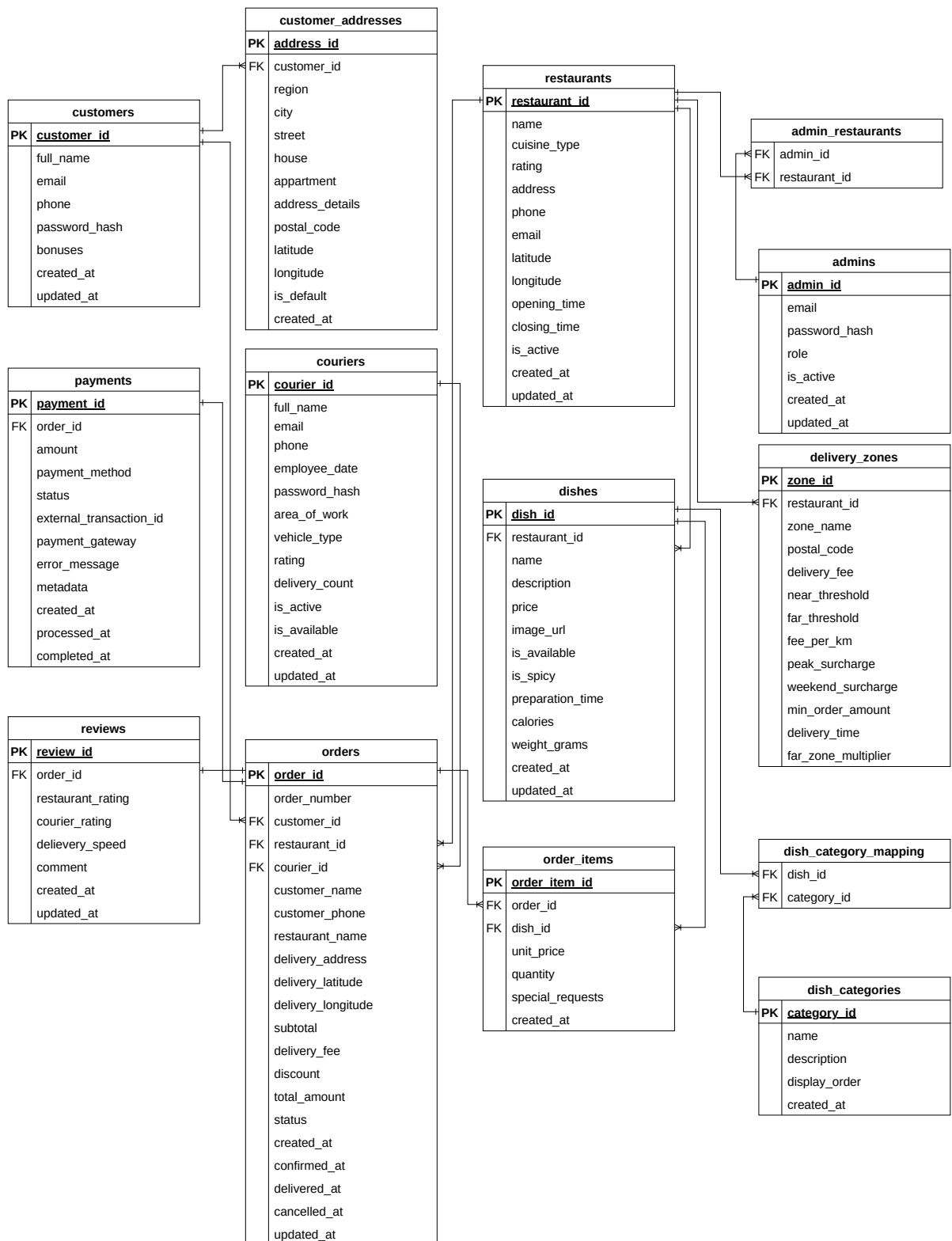


Рисунок 2.1 – Диаграмма базы данных

База данных приведена к третьей нормальной форме: все атрибуты атомарны, все таблицы имеют простые первичные ключи (исключена частичная зависимость), отсутствуют транзитивные зависимости неключе-

вых атрибутов. Денормализация в таблицах `orders` (поля `customer_name`, `restaurant_name`, `delivery_address`) и `order_items` (поля `dish_name` и `unit_price`) применена для сохранения состояния данных на момент создания заказа.

2.2 Проектирование функции и триггеров

На рисунке 2.2 представлен триггер для обновления рейтинга ресторана после добавления отзыва.

На рисунке 2.3 представлен триггер для обновления рейтинга курьера после добавления отзыва.

На рисунках 2.4 и 2.5 представлена функция для расчёта итоговой суммы заказа с учётом всех факторов скидок и наценок.

Алгоритм триггера обновления рейтинга ресторана. Срабатывает после INSERT или UPDATE в reviews

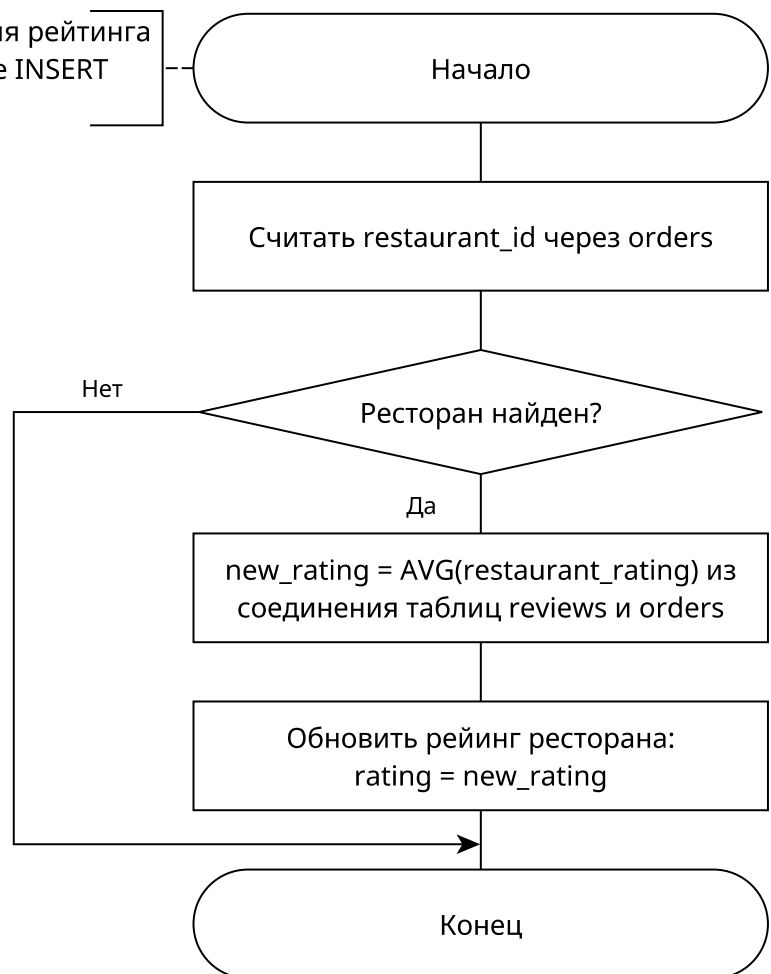


Рисунок 2.2 – Триггер для обновления рейтинга ресторана после добавления отзыва

Алгоритм триггера обновления рейтинга курьера. Срабатывает после INSERT или UPDATE в reviews

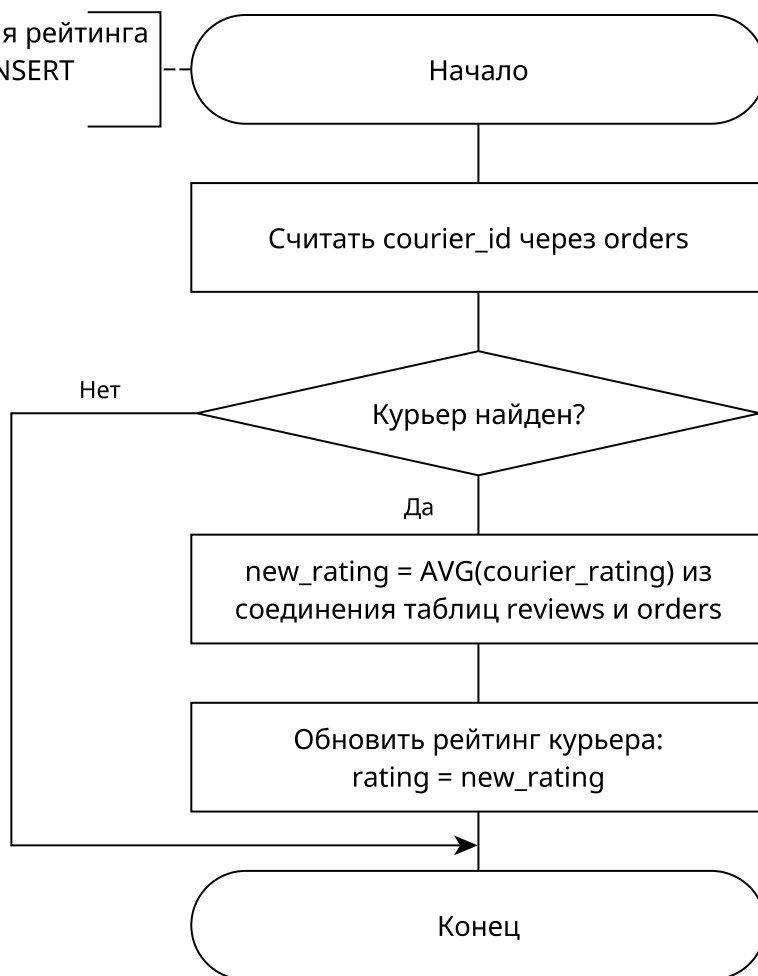


Рисунок 2.3 – Триггер для обновления рейтинга курьера после добавления отзыва

Алгоритм функции расчёта итоговой цены, с учётом всех факторов.
Вход: номер заказа и процент скидки на первый заказ `p_discount_rate`.
Выход: рассчитанная цена заказа.

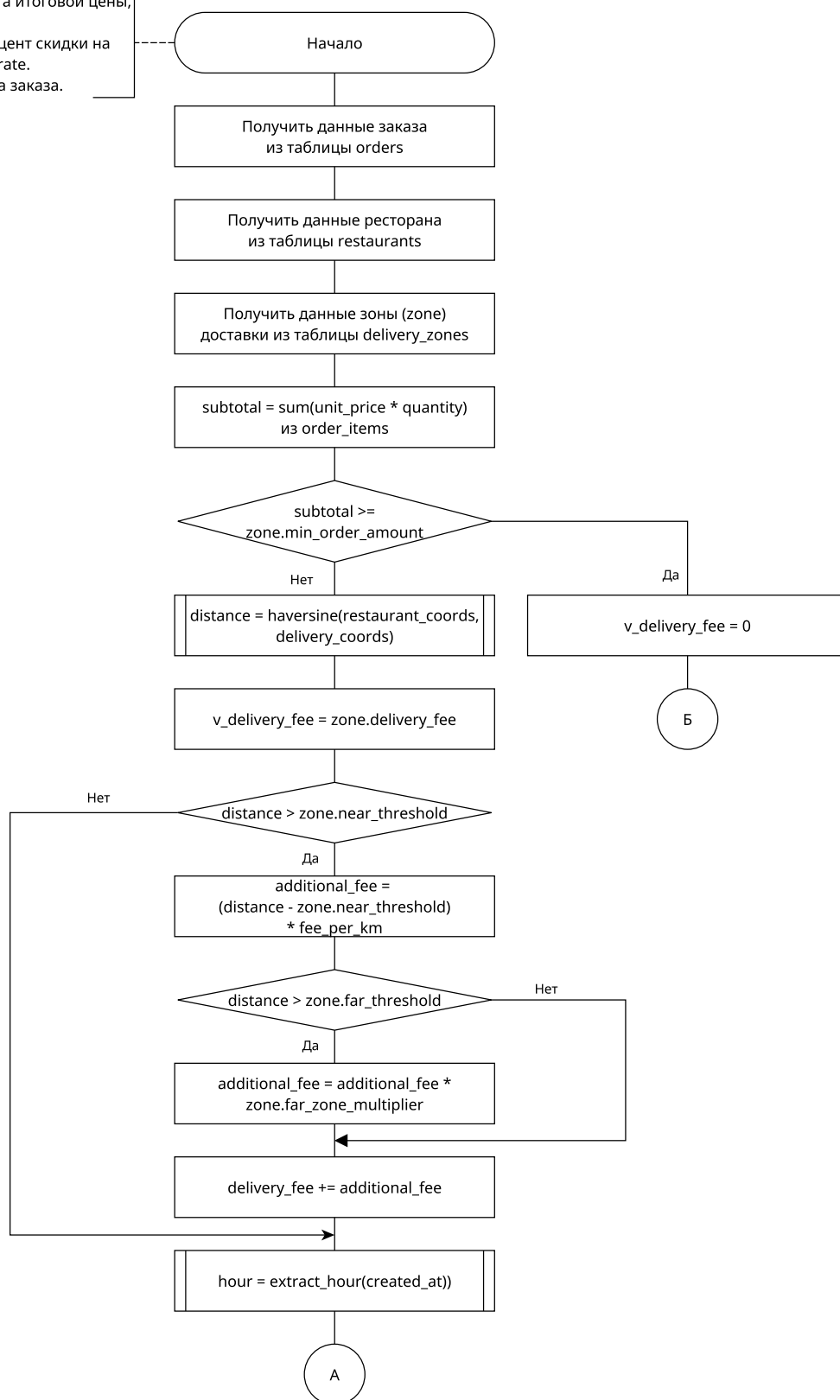


Рисунок 2.4 – Функция подсчёта итоговой суммы заказа. Часть 1

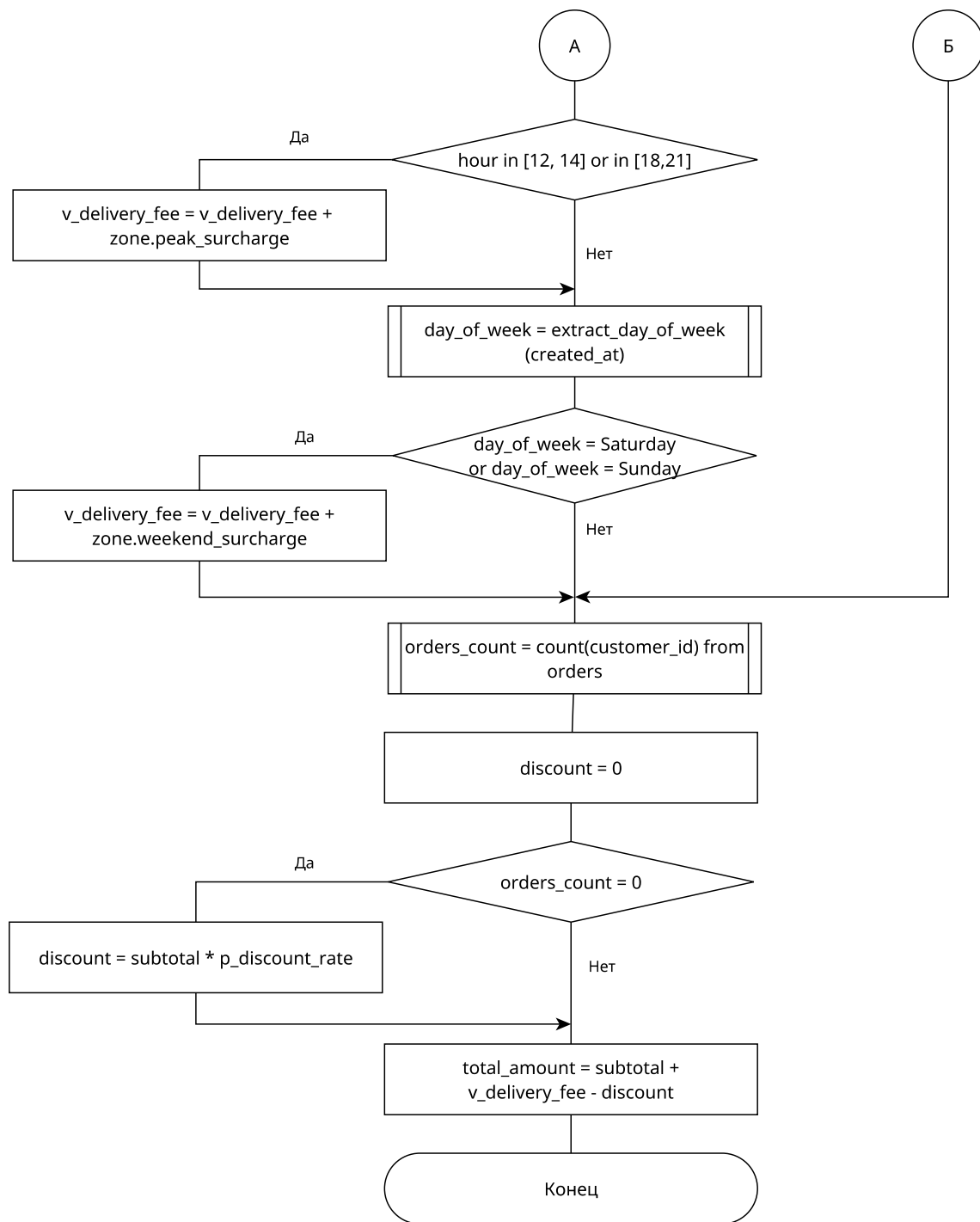


Рисунок 2.5 – Функция подсчёта итоговой суммы заказа. Часть 2

2.3 Проектирование ролевой модели

В соответствии с требованиями к системе разграничения доступа были спроектированы следующие роли: роль клиента, роль администратора ресторана, роль курьера и роль администратора системы.

Роль «Клиент» предназначена для пользователей, оформляющих

заказы в системе. Данная роль имеет следующие права доступа:

- чтение данных из таблиц **restaurants**, **dish_categories** и **dishes** для просмотра доступных ресторанов и меню;
- чтение и изменение собственных данных в таблице **customers**;
- чтение, вставка и изменение собственных записей в таблице **customer_addresses** для управления адресами доставки;
- чтение и вставка записей в таблицу **orders** для создания новых заказов (доступ ограничен только заказами данного клиента);
- чтение и вставка записей в таблицу **order_items** для формирования состава заказа;
- чтение данных из таблицы **payments** для просмотра информации об оплате своих заказов;
- чтение данных из таблицы **delivery_zones** для проверки доступности доставки по адресу клиента.
- чтение, вставка и изменение собственных записей в таблице **reviews** для оставления отзывов о заказах.

Роль «Администратор ресторана» предназначена для управления рестораном и его меню. Данная роль имеет следующие права доступа:

- чтение и изменение данных в таблице **restaurants** для обновления информации о своём ресторане;
- полный доступ (чтение, вставка, изменение, удаление) к таблицам **dish_categories** и **dishes** для управления меню своего ресторана;
- чтение и изменение записей в таблице **orders** для обработки заказов своего ресторана;
- чтение данных из таблицы **order_items** для просмотра состава заказов;
- полный доступ к таблице **delivery_zones** для управления зонами доставки своего ресторана.
- чтение данных из таблицы **reviews** для анализа отзывов о своём ресторане.

Роль «Курьер» предназначена для сотрудников службы доставки. Данная роль имеет следующие права доступа:

- чтение и изменение собственных данных в таблице **couriers**;
- чтение и изменение записей в таблице **orders** для обновления

статуса доставки назначенных ему заказов;

- чтение данных из таблицы **order_items** для просмотра состава доставляемого заказа;
- чтение данных из таблиц **restaurants**, **customer_addresses** и **customers** для получения информации о точках отправления и назначения;
- чтение данных из таблицы **delivery_zones** для уточнения границ доставки.
- чтение данных из таблицы **reviews** для просмотра оценок своей работы.

Роль «Администратор системы» предназначена для технических специалистов, обслуживающих систему. Данная роль имеет следующие права доступа:

- полный доступ ко всем таблицам базы данных для администрирования и устранения технических проблем;
- права на выполнение всех хранимых функций и процедур;
- доступ к системным объектам базы данных.

2.4 Проектирование приложения

Структура приложения построена на основе многослойной модели, включающей уровень представления для взаимодействия с клиентскими приложениями, уровень бизнес-логики для реализации бизнес-правил и уровень доступа к данным для работы с БД.

Чтобы снизить нагрузку на основную базу данных, на стыке бизнес-логики и работы с данными спроектирован механизм кэширования. В кэш отправляются данные, которые пользователи запрашивают часто, но меняются они редко: информация о ресторанах, меню и зонах доставки.

На рисунке 2.6 представлена диаграмма последовательности взаимодействия приложения с реляционной СУБД и In-Memory СУБД.

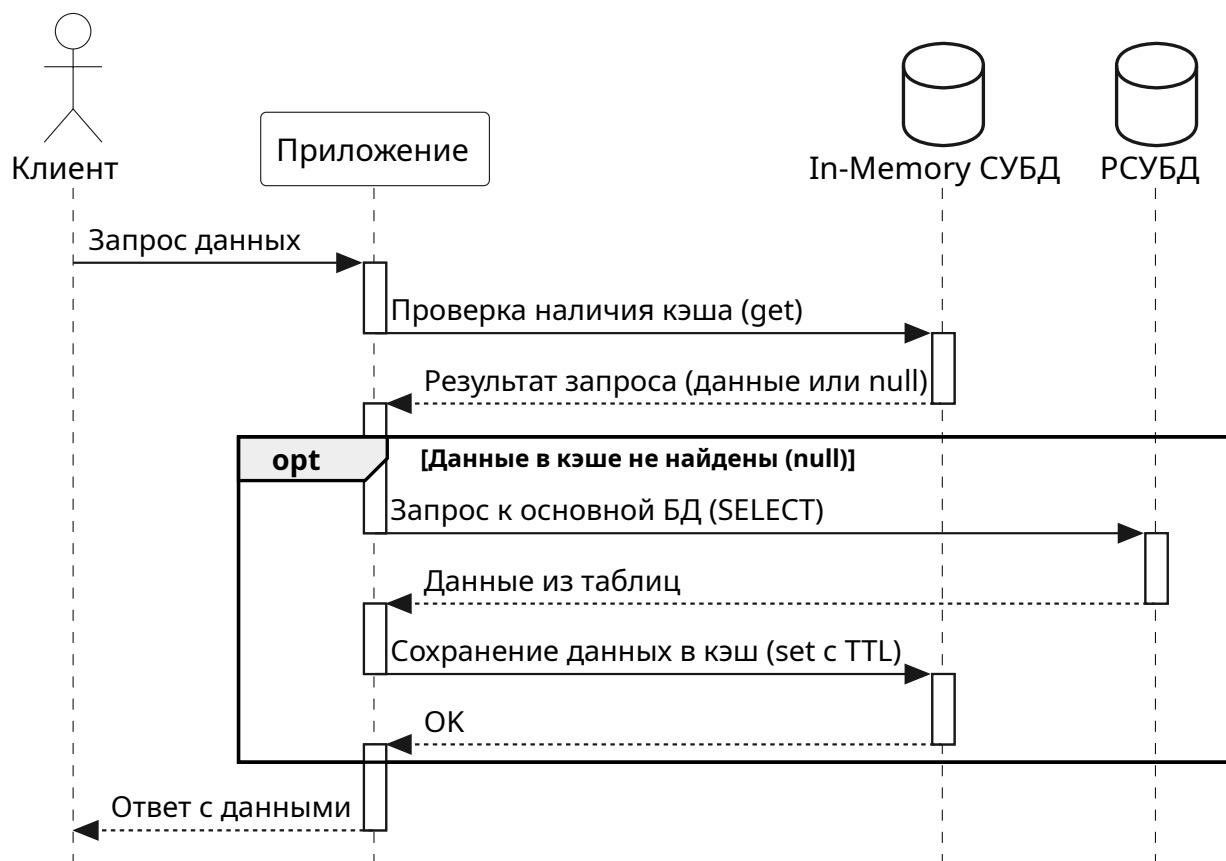


Рисунок 2.6 – Диаграмма взаимодействия приложения с реляционной СУБД и с In-Memory СУБД

При поступлении запроса данных приложение сначала обращается к In-Memory СУБД. В случае cache hit данные возвращаются клиенту без обращения к реляционной СУБД. В случае cache miss выполняется запрос к реляционной СУБД, результат сохраняется в кэш с заданным временем жизни и возвращается клиенту. При изменении данных старый кэш удаляется с помощью аннотации, чтобы пользователь не получил устаревшую информацию.

На рисунке 2.7 представлена диаграмма пакетов проектируемого приложения.

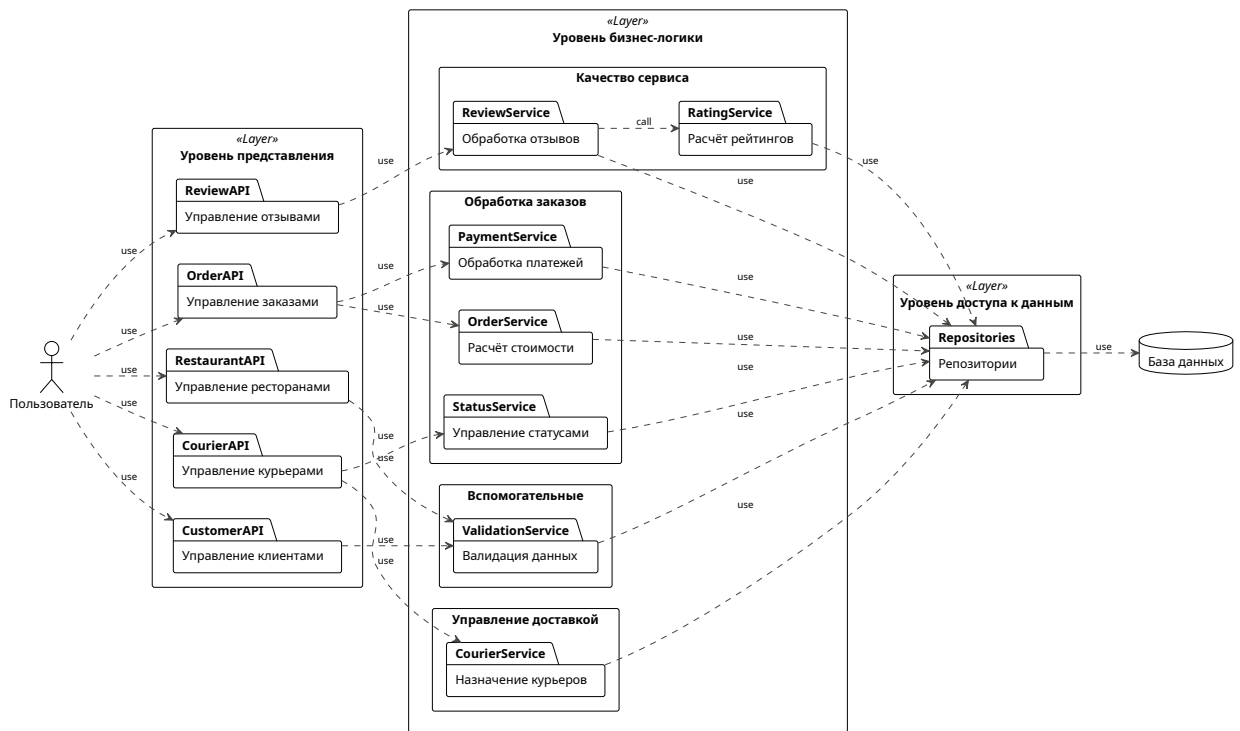


Рисунок 2.7 – Диаграмма пакетов приложения

2.5 Вывод

В конструкторском разделе были спроектированы база данных и ограничения её целостности. Обосновано соответствие таблиц третьей нормальной форме (3НФ). Составлен алгоритм функции для расчёта итоговой суммы заказа с учётом скидок и наценок. Описана ролевая модель доступа на уровне базы данных и спроектировано приложение для взаимодействия с базой данных

3 Технологический раздел

3.1 Выбор средств реализации базы данных

В рамках реляционной модели данных для выбора СУБД для сравнительного анализа были выбраны следующие системы: PostgreSQL [12], MySQL [13], Microsoft SQL Server [14] и Oracle Database [15].

Для сравнения определены следующие критерии:

- открытый исходный код (Да/Нет);
- стоимость (бесплатно/платно);
- поддержка ACID (Да/Нет);
- поддержка JSON (Да/Нет);
- репликация данных (Да/Нет).

В таблице 3.1 представлены результаты сравнения выбранных СУБД по указанным критериям.

Таблица 3.1 – Сравнительный анализ СУБД

Критерий	СУБД			
	PostgreSQL	MySQL	Microsoft SQL Server	Oracle Database
Открытый исходный код	Да	Да	Нет	Нет
Стоимость	Бесплатно	Бесплатно	Платно	Платно
Поддержка ACID	Да	Да	Да	Да
Поддержка JSON	Да	Да	Да	Да
Репликация	Да	Да	Да	Да

На основе проведённого анализа для реализации базы данных выбрана СУБД PostgreSQL. Все рассмотренные системы поддерживают ACID, JSON и репликацию, однако PostgreSQL и MySQL являются единственными бесплатными решениями с открытым исходным кодом. В отличие от MySQL, лицензия которого накладывает ограничения при распространении коммерческого программного обеспечения, PostgreSQL распространяется под собственной свободной лицензией без подобных условий. Таким образом, PostgreSQL является оптимальным выбором для разрабатываемой системы.

3.2 Выбор In-Memory СУБД

Для организации временного хранения и кэширования данных приложения выбрана NoSQL СУБД Redis [16]. Она хранит данные в оперативной памяти в формате «ключ-значение».

3.3 Выбор средств реализации приложения

Для реализации приложения доступа к базе данных выбран язык программирования Java [17]. Java является строго типизированным объектно-ориентированным языком, что обеспечивает надёжность и сопровождаемость кода.

В качестве фреймворка используется Spring Boot [18], который предоставляет автоконфигурацию и встроенный сервер приложений, что существенно сокращает объём инфраструктурного кода и упрощает развёртывание приложения.

Для взаимодействия с БД применяется Spring Data JPA [19] — модуль Spring, обеспечивающий объектно-реляционное отображение и позволяющий работать с данными через репозитории без написания SQL-запросов вручную.

Для интеграции кэширования данных используется пакет Spring Data Redis [20], обеспечивающий взаимодействие со структурами данных Redis.

3.4 Создание таблиц и типов базы данных

В листингах 3.1–3.5 представлены SQL-запросы для создания типов базы данных.

Листинг 3.1 – Создание типа `order_status`

```
CREATE TYPE order_status AS ENUM (  
    'pending',  
    'confirmed',  
    'preparing',  
    'ready',  
    'delivering',  
    'delivered',  
    'cancelled'  
);
```

Листинг 3.2 – Создание типа payment_method

```
CREATE TYPE payment_method AS ENUM (  
    'cash',  
    'card',  
    'online',  
    'bonuses'  
);
```

Листинг 3.3 – Создание типа payment_status

```
CREATE TYPE payment_status AS ENUM (  
    'pending',  
    'processing',  
    'completed',  
    'failed',  
    'refunded');
```

Листинг 3.4 – Создание типа vehicle_type

```
CREATE TYPE vehicle_type AS ENUM (  
    'car',  
    'bicycle',  
    'walking');
```

Листинг 3.5 – Создание типа admin_role

```
CREATE TYPE admin_role AS ENUM (  
    'system_admin',  
    'restaurant_admin'  
);
```

В листингах 3.6–3.19 представлены SQL-запросы для создания таблиц базы данных.

Листинг 3.6 – Создание таблицы customers

```
CREATE TABLE IF NOT EXISTS customers  
(  
    customer_id    UUID,  
    full_name      VARCHAR(150),  
    email          VARCHAR(255),  
    phone          VARCHAR(20),  
    password_hash  VARCHAR(255),  
    bonuses        INTEGER,  
    is_active      BOOLEAN,  
    created_at     TIMESTAMP,  
    updated_at     TIMESTAMP  
);
```

Листинг 3.7 – Создание таблицы customer_addresses

```
CREATE TABLE IF NOT EXISTS customer_addresses
(
    address_id        UUID,
    customer_id       UUID,
    region            VARCHAR(100),
    city              VARCHAR(100),
    street            VARCHAR(200),
    house             VARCHAR(20),
    apartment         VARCHAR(20),
    address_details   TEXT,
    postal_code       VARCHAR(20),
    latitude           DECIMAL(12, 8),
    longitude         DECIMAL(12, 8),
    is_default        BOOLEAN,
    created_at        TIMESTAMP);
```

Листинг 3.8 – Создание таблицы couriers

```
CREATE TABLE IF NOT EXISTS couriers
(
    courier_id        UUID,
    full_name         VARCHAR(75),
    phone             VARCHAR(20),
    email             VARCHAR(255),
    password_hash     VARCHAR(255),
    employee_date     DATE,
    area_of_work      VARCHAR(100),
    vehicle_type      vehicle_type,
    rating            DECIMAL(3, 2),
    is_available      BOOLEAN,
    is_active         BOOLEAN,
    created_at        TIMESTAMP,
    updated_at        TIMESTAMP
);
```

Листинг 3.9 – Создание таблицы admins

```
CREATE TABLE IF NOT EXISTS admins
(
    admin_id          UUID,
    email             VARCHAR(255),
    password_hash     VARCHAR(255),
    role              admin_role,
    is_active         BOOLEAN,
    created_at        TIMESTAMP,
    updated_at        TIMESTAMP
);
```

Листинг 3.10 – Создание таблицы admin_restaurants

```
CREATE TABLE IF NOT EXISTS admin_restaurants
(
    admin_id      UUID,
    restaurant_id UUID
);
```

Листинг 3.11 – Создание таблицы restaurants

```
CREATE TABLE IF NOT EXISTS restaurants
(
    restaurant_id UUID,
    name           VARCHAR(75),
    cuisine_type   VARCHAR(25),
    rating         DECIMAL(4, 2),
    address        TEXT,
    phone          VARCHAR(20),
    email          VARCHAR(255),
    latitude       DECIMAL(12, 8),
    longitude      DECIMAL(12, 8),
    opening_time   TIME,
    closing_time   TIME,
    is_active      BOOLEAN,
    created_at     TIMESTAMP,
    updated_at     TIMESTAMP
);
```

Листинг 3.12 – Создание таблицы dish_categories

```
CREATE TABLE IF NOT EXISTS dish_categories
(
    category_id   UUID,
    name          VARCHAR(100),
    description    TEXT,
    display_order INTEGER,
    created_at    TIMESTAMP
);
```

Листинг 3.13 – Создание таблицы delivery_zones

```
CREATE TABLE IF NOT EXISTS delivery_zones
(
    zone_id          UUID,
    restaurant_id    UUID,
    zone_name        VARCHAR,
    postal_code      VARCHAR,
    delivery_fee     DECIMAL(10, 2),
    near_threshold   DECIMAL(10, 2),
    far_threshold    DECIMAL(10, 2),

```

```

    far_zone_multiplier DECIMAL(10, 2),
    fee_per_km          DECIMAL(10, 2),
    peak_surcharge      DECIMAL(10, 2),
    weekend_surcharge    DECIMAL(10, 2),
    min_order_amount    DECIMAL(10, 2),
    delivery_time       INTEGER
);

```

Листинг 3.14 – Создание таблицы orders

```

CREATE TABLE IF NOT EXISTS orders
(
    order_id          UUID,
    order_number      VARCHAR(20),
    customer_id       UUID,
    restaurant_id     UUID,
    courier_id        UUID,
    customer_name      VARCHAR(150),
    customer_phone     VARCHAR(20),
    restaurant_name    VARCHAR(150),
    delivery_address   TEXT,
    delivery_latitude  DECIMAL(12, 8),
    delivery_longitude DECIMAL(12, 8),
    subtotal           DECIMAL(10, 2),
    delivery_fee       DECIMAL(10, 2),
    discount           DECIMAL(10, 2),
    total_amount       DECIMAL(10, 2),
    status             order_status,
    created_at         TIMESTAMP,
    confirmed_at       TIMESTAMP,
    delivered_at       TIMESTAMP,
    cancelled_at       TIMESTAMP,
    updated_at         TIMESTAMP
);

```

Листинг 3.15 – Создание таблицы dishes

```

CREATE TABLE IF NOT EXISTS dishes (
    dish_id          UUID,
    restaurant_id    UUID,
    name              VARCHAR(150),
    description       TEXT,
    price             DECIMAL(10, 2),
    image_url         TEXT,
    is_available      BOOLEAN,
    is_spicy          BOOLEAN,
    preparation_time  INTEGER,
    calories          INTEGER,
    weight_grams      INTEGER,

```

```
created_at      TIMESTAMP ,
updated_at      TIMESTAMP );
```

Листинг 3.16 – Создание таблицы order_items

```
CREATE TABLE IF NOT EXISTS order_items
(
    order_item_id    UUID ,
    order_id         UUID ,
    dish_id          UUID ,
    dish_name        VARCHAR(150) ,
    unit_price       DECIMAL(10, 2) ,
    quantity         INTEGER ,
    special_requests TEXT ,
    created_at       TIMESTAMP
);
```

Листинг 3.17 – Создание таблицы payments

```
CREATE TABLE IF NOT EXISTS payments (
    payment_id      UUID ,
    order_id        UUID ,
    amount          DECIMAL(10, 2) ,
    payment_method  payment_method ,
    status          payment_status ,
    external_transaction_id VARCHAR(255) ,
    payment_gateway VARCHAR(50) ,
    error_message   TEXT ,
    metadata        JSONB ,
    created_at      TIMESTAMP ,
    processed_at    TIMESTAMP ,
    completed_at    TIMESTAMP
);
```

Листинг 3.18 – Создание таблицы reviews

```
CREATE TABLE IF NOT EXISTS reviews
(
    review_id      UUID ,
    order_id       UUID ,
    restaurant_rating INTEGER ,
    courier_rating  INTEGER ,
    delivery_speed  INTEGER ,
    comment        TEXT ,
    created_at     TIMESTAMP ,
    updated_at     TIMESTAMP
);
```


Листинг 3.19 – Создание таблицы dish_category_mapping

```
CREATE TABLE IF NOT EXISTS dish_category_mapping
(
    dish_id      UUID,
    category_id  UUID
);
```

3.5 Реализация ограничений целостности

В листингах 3.20–?? представлены SQL-запросы для добавления ограничений целостности в реализованные таблицы.

Листинг 3.20 – Описание ограничений целостности для таблицы customers

```
ALTER TABLE customers
    ADD CONSTRAINT add_customers_primary_key PRIMARY KEY (customer_id),
    ALTER COLUMN customer_id SET DEFAULT gen_random_uuid(),
    ALTER COLUMN customer_id SET NOT NULL,
    ALTER COLUMN full_name SET NOT NULL,
    ALTER COLUMN phone SET NOT NULL,
    ADD CONSTRAINT customers_phone_unique UNIQUE (phone),
    ALTER COLUMN email SET NOT NULL,
    ADD CONSTRAINT customers_email_unique UNIQUE (email),
    ADD CONSTRAINT check_positives_bonuses CHECK (bonuses >= 0),
    ALTER COLUMN bonuses SET DEFAULT 0,
    ALTER COLUMN password_hash SET NOT NULL,
    ALTER COLUMN is_active SET DEFAULT TRUE,
    ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP,
    ALTER COLUMN updated_at SET DEFAULT CURRENT_TIMESTAMP;
```

Листинг 3.21 – Описание ограничений целостности для таблицы customer_addresses

```
ALTER TABLE customer_addresses
    ADD CONSTRAINT add_customer_addresses_primary_key PRIMARY KEY
        (address_id),
    ALTER COLUMN address_id SET DEFAULT gen_random_uuid(),
    ALTER COLUMN address_id SET NOT NULL,
    ADD CONSTRAINT address_customer_id_fkey FOREIGN KEY (customer_id)
        REFERENCES customers (customer_id) ON DELETE CASCADE,
    ALTER COLUMN customer_id SET NOT NULL,
    ALTER COLUMN city SET NOT NULL,
    ALTER COLUMN street SET NOT NULL,
    ALTER COLUMN house SET NOT NULL,
    ALTER COLUMN is_default SET DEFAULT false,
    ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP;
```

Листинг 3.22 – Описание ограничений целостности для таблицы couriers

```
ALTER TABLE couriers
  ADD CONSTRAINT add_couriers_primary_key PRIMARY KEY (courier_id),
  ALTER COLUMN courier_id SET DEFAULT gen_random_uuid(),
  ALTER COLUMN courier_id SET NOT NULL,
  ALTER COLUMN full_name SET NOT NULL,
  ALTER COLUMN phone SET NOT NULL,
  ADD CONSTRAINT couriers_phone_unique UNIQUE (phone),
  ALTER COLUMN email SET NOT NULL,
  ADD CONSTRAINT couriers_email_unique UNIQUE (email),
  ALTER COLUMN password_hash SET NOT NULL,
  ALTER COLUMN employee_date SET NOT NULL,
  ALTER COLUMN vehicle_type SET NOT NULL,
  ADD CONSTRAINT check_rating CHECK (rating >= 0 AND rating <= 5),
  ALTER COLUMN rating SET DEFAULT 0,
  ALTER COLUMN is_active SET DEFAULT true,
  ALTER COLUMN is_available SET DEFAULT true,
  ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP,
  ALTER COLUMN updated_at SET DEFAULT CURRENT_TIMESTAMP;
```

Листинг 3.23 – Описание ограничений целостности для таблицы admins

```
ALTER TABLE admins
  ADD CONSTRAINT add_admins_primary_key PRIMARY KEY (admin_id),
  ALTER COLUMN admin_id SET DEFAULT gen_random_uuid(),
  ALTER COLUMN admin_id SET NOT NULL,
  ALTER COLUMN email SET NOT NULL,
  ADD CONSTRAINT admins_email_unique UNIQUE (email),
  ALTER COLUMN password_hash SET NOT NULL,
  ALTER COLUMN role SET NOT NULL,
  ALTER COLUMN is_active SET DEFAULT TRUE,
  ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP,
  ALTER COLUMN updated_at SET DEFAULT CURRENT_TIMESTAMP;
```

Листинг 3.24 – Описание ограничений целостности для таблицы restaurants

```
ALTER TABLE restaurants
  ADD CONSTRAINT add_restaurants_primary_key PRIMARY KEY
    (restaurant_id),
  ALTER COLUMN restaurant_id SET DEFAULT gen_random_uuid(),
  ALTER COLUMN restaurant_id SET NOT NULL,
  ALTER COLUMN name SET NOT NULL,
  ADD CONSTRAINT check_rating CHECK (rating >= 0 AND rating <= 5),
  ALTER COLUMN rating SET DEFAULT 0.00,
  ALTER COLUMN address SET NOT NULL,
  ALTER COLUMN phone SET NOT NULL,
  ADD CONSTRAINT restaurants_phone_unique UNIQUE (phone),
  ALTER COLUMN email SET NOT NULL,
  ADD CONSTRAINT restaurants_email_unique UNIQUE (email),
```

```
ALTER COLUMN is_active SET DEFAULT true,
ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP,
ALTER COLUMN updated_at SET DEFAULT CURRENT_TIMESTAMP;
```

Листинг 3.25 – Описание ограничений целостности для таблицы
admin_restaurants

```
ALTER TABLE admin_restaurants
  ADD CONSTRAINT admin_restaurants_admin_id_fkey FOREIGN KEY (admin_id)
    REFERENCES admins (admin_id) ON DELETE CASCADE,
  ALTER COLUMN admin_id SET NOT NULL,
  ADD CONSTRAINT admin_restaurants_restaurant_id_fkey FOREIGN KEY
    (restaurant_id) REFERENCES restaurants (restaurant_id) ON DELETE
    CASCADE,
  ALTER COLUMN restaurant_id SET NOT NULL,
  ADD CONSTRAINT add_admin_restaurants_pk PRIMARY KEY (admin_id,
    restaurant_id);
```

Листинг 3.26 – Описание ограничений целостности для таблицы
delivery_zones

```
ALTER TABLE delivery_zones
  ADD CONSTRAINT add_delivery_zones_pk PRIMARY KEY (zone_id),
  ALTER COLUMN zone_id SET DEFAULT gen_random_uuid(),
  ALTER COLUMN zone_id SET NOT NULL,
  ADD CONSTRAINT restaurant_id_fkey FOREIGN KEY (restaurant_id)
    REFERENCES restaurants (restaurant_id) ON DELETE CASCADE,
  ALTER COLUMN restaurant_id SET NOT NULL,
  ALTER COLUMN zone_name SET NOT NULL,
  ALTER COLUMN postal_code SET NOT NULL,
  ADD CONSTRAINT check_delivery_fee_positive CHECK (delivery_fee >= 0),
  ALTER COLUMN delivery_fee SET DEFAULT 0,
  ADD CONSTRAINT check_near_threshold_positive CHECK (near_threshold >=
    0),
  ALTER COLUMN near_threshold SET DEFAULT 3,
  ADD CONSTRAINT check_far_threshold_positive CHECK (far_threshold >=
    0),
  ALTER COLUMN far_threshold SET DEFAULT 5,
  ADD CONSTRAINT check_far_greater_than_near CHECK (far_threshold >=
    near_threshold),
  ADD CONSTRAINT check_far_zone_multiplier_positive CHECK
    (far_zone_multiplier > 0),
  ALTER COLUMN far_zone_multiplier SET DEFAULT 1.5,
  ADD CONSTRAINT check_fee_per_km_positive CHECK (fee_per_km >= 0),
  ALTER COLUMN fee_per_km SET DEFAULT 0,
  ADD CONSTRAINT check_peak_surcharge_positive CHECK (peak_surcharge >=
    0),
  ALTER COLUMN peak_surcharge SET DEFAULT 0,
```

```

ADD CONSTRAINT check_weekend_surcharge_positive CHECK
    (weekend_surcharge >= 0),
ALTER COLUMN weekend_surcharge SET DEFAULT 0,
ADD CONSTRAINT check_min_order_amount_positive CHECK
    (min_order_amount >= 0),
ALTER COLUMN min_order_amount SET DEFAULT 0,
ADD CONSTRAINT check_delivery_time_positive CHECK (delivery_time >=
    0);

```

Листинг 3.27 – Описание ограничений целостности для таблицы dish_categories

```

ALTER TABLE dish_categories
    ADD CONSTRAINT add_dish_ctgrs_primary_key PRIMARY KEY (category_id),
    ALTER COLUMN category_id SET DEFAULT gen_random_uuid(),
    ALTER COLUMN category_id SET NOT NULL,
    ALTER COLUMN name SET NOT NULL,
    ALTER COLUMN display_order SET DEFAULT 0,
    ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP;

```

Листинг 3.28 – Описание ограничений целостности для таблицы dishes

```

ALTER TABLE dishes
    ADD CONSTRAINT add_dishes_primary_key PRIMARY KEY (dish_id),
    ALTER COLUMN dish_id SET DEFAULT gen_random_uuid(),
    ALTER COLUMN dish_id SET NOT NULL,
    ADD CONSTRAINT dishes_restaurant_id_fkey FOREIGN KEY (restaurant_id)
        REFERENCES restaurants (restaurant_id) ON DELETE CASCADE,
    ALTER COLUMN restaurant_id SET NOT NULL,
    ALTER COLUMN name SET NOT NULL,
    ALTER COLUMN price SET NOT NULL,
    ADD CONSTRAINT check_price_positive CHECK (price > 0),
    ADD CONSTRAINT check_prep_time_positive CHECK (preparation_time >= 0),
    ADD CONSTRAINT check_calories_positive CHECK (calories >= 0),
    ADD CONSTRAINT check_weight_positive CHECK (weight_grams >= 0),
    ALTER COLUMN is_available SET DEFAULT true,
    ALTER COLUMN is_spicy SET DEFAULT false,
    ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP,
    ALTER COLUMN updated_at SET DEFAULT CURRENT_TIMESTAMP;

```

Листинг 3.29 – Описание ограничений целостности для таблицы dish_category_mapping

```

ALTER TABLE dish_category_mapping
    ADD CONSTRAINT dish_id_fkey FOREIGN KEY (dish_id) REFERENCES dishes
        (dish_id),
    ALTER COLUMN dish_id SET NOT NULL,
    ADD CONSTRAINT category_id_fkey FOREIGN KEY (category_id) REFERENCES
        dish_categories (category_id),

```

```

ALTER COLUMN category_id SET NOT NULL,
ADD CONSTRAINT add_dish_category_mapping_pk PRIMARY KEY (dish_id,
category_id);

```

Листинг 3.30 – Описание ограничений целостности для таблицы order_items

```

ALTER TABLE order_items
ADD CONSTRAINT add_order_items_primary_key PRIMARY KEY
(order_item_id),
ALTER COLUMN order_item_id SET DEFAULT gen_random_uuid(),
ALTER COLUMN order_item_id SET NOT NULL,
ADD CONSTRAINT order_items_order_id_fkey FOREIGN KEY (order_id)
REFERENCES orders (order_id) ON DELETE CASCADE,
ALTER COLUMN order_id SET NOT NULL,
ADD CONSTRAINT order_items_dish_id_fkey FOREIGN KEY (dish_id)
REFERENCES dishes (dish_id) ON DELETE RESTRICT,
ALTER COLUMN dish_id SET NOT NULL,
ALTER COLUMN unit_price SET NOT NULL,
ADD CONSTRAINT check_unit_price_positive CHECK (unit_price > 0),
ALTER COLUMN quantity SET NOT NULL,
ADD CONSTRAINT check_quantity_positive CHECK (quantity > 0),
ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP;

```

Листинг 3.31 – Описание ограничений целостности для таблицы orders

```

ALTER TABLE orders
ADD CONSTRAINT add_orders_primary_key PRIMARY KEY (order_id),
ALTER COLUMN order_id SET DEFAULT gen_random_uuid(),
ALTER COLUMN order_id SET NOT NULL,
ADD CONSTRAINT orders_order_number_unique UNIQUE (order_number),
ALTER COLUMN order_number SET NOT NULL,
ADD CONSTRAINT orders_customer_id_fkey FOREIGN KEY (customer_id)
REFERENCES customers (customer_id) ON DELETE RESTRICT,
ALTER COLUMN customer_id SET NOT NULL,
ADD CONSTRAINT orders_restaurant_id_fkey FOREIGN KEY (restaurant_id)
REFERENCES restaurants (restaurant_id) ON DELETE RESTRICT,
ALTER COLUMN restaurant_id SET NOT NULL,
ADD CONSTRAINT orders_courier_id_fkey FOREIGN KEY (courier_id)
REFERENCES couriers (courier_id) ON DELETE SET NULL,
ALTER COLUMN subtotal SET NOT NULL,
ADD CONSTRAINT check_subtotal_positive CHECK (subtotal >= 0),
ALTER COLUMN delivery_fee SET DEFAULT 0,
ADD CONSTRAINT check_delivery_fee_positive CHECK (delivery_fee >= 0),
ALTER COLUMN discount SET DEFAULT 0,
ADD CONSTRAINT check_discount_positive CHECK (discount >= 0),
ALTER COLUMN total_amount SET NOT NULL,
ADD CONSTRAINT check_total_amount_positive CHECK (total_amount >= 0),
ALTER COLUMN status SET NOT NULL,
ALTER COLUMN status SET DEFAULT 'pending',

```

```
ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP ,
ALTER COLUMN updated_at SET DEFAULT CURRENT_TIMESTAMP;
```

Листинг 3.32 – Описание ограничений целостности для таблицы payments

```
ALTER TABLE payments
ADD CONSTRAINT add_payments_primary_key PRIMARY KEY (payment_id),
ALTER COLUMN payment_id SET DEFAULT gen_random_uuid(),
ALTER COLUMN payment_id SET NOT NULL,
ADD CONSTRAINT payments_order_id_fkey FOREIGN KEY (order_id)
REFERENCES orders (order_id) ON DELETE RESTRICT,
ALTER COLUMN order_id SET NOT NULL,
ADD CONSTRAINT payments_order_id_unique UNIQUE (order_id),
ALTER COLUMN amount SET NOT NULL,
ADD CONSTRAINT check_amount_positive CHECK (amount > 0),
ALTER COLUMN payment_method SET NOT NULL,
ALTER COLUMN status SET DEFAULT 'pending',
ALTER COLUMN status SET NOT NULL,
ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP;
```

Листинг 3.33 – Описание ограничений целостности для таблицы reviews

```
ALTER TABLE reviews
ADD CONSTRAINT add_reviews_primary_key PRIMARY KEY (review_id),
ALTER COLUMN review_id SET DEFAULT gen_random_uuid(),
ALTER COLUMN review_id SET NOT NULL,
ADD CONSTRAINT reviews_order_id_fkey FOREIGN KEY (order_id)
REFERENCES orders (order_id) ON DELETE CASCADE,
ALTER COLUMN order_id SET NOT NULL,
ADD CONSTRAINT reviews_order_id_unique UNIQUE (order_id),
ADD CONSTRAINT check_restaurant_rating CHECK (restaurant_rating >= 1
AND restaurant_rating <= 5),
ADD CONSTRAINT check_courier_rating CHECK (courier_rating >= 1 AND
courier_rating <= 5),
ADD CONSTRAINT check_delivery_speed CHECK (delivery_speed >= 1 AND
delivery_speed <= 5),
ALTER COLUMN created_at SET DEFAULT CURRENT_TIMESTAMP,
ALTER COLUMN updated_at SET DEFAULT CURRENT_TIMESTAMP;
```

3.6 Создание функции и триггеров

В приложениях Б.1–Б.3 представлена функция для создания функции подсчёта итоговой суммы заказа.

В приложениях Б.4 и Б.5 представлены функции и триггеры для автоматического обновления рейтинга ресторана и курьера после добавления или изменения отзыва.

3.7 Тестирование разработанных функций

Для тестирования функции `calculate_order_total` и триггеров обновления рейтингов выделены классы эквивалентности входных данных. В таблицах 3.2–3.4 представлены классы эквивалентности и результаты тестирования.

Таблица 3.2 – Классы эквивалентности для функции `calculate_order_total`

№	Класс эквивалентности	Ожидаемый результат	Фактический результат
1	Корректный <code>order_id</code> , <code>subtotal</code> ниже порога бесплатной доставки, расстояние в ближней зоне, обычное время	Корректный расчёт <code>subtotal</code> , <code>delivery_fee</code> с надбавкой за расстояние, <code>discount</code> = 0	Совпадает с ожидаемым
2	Корректный <code>order_id</code> , <code>subtotal</code> не ниже минимальной суммы для бесплатной доставки	<code>delivery_fee</code> = 0, <code>discount</code> = 0	Совпадает с ожидаемым
3	Корректный <code>order_id</code> , расстояние в дальней зоне	<code>delivery_fee</code> с надбавкой, умноженной на <code>far_zone_multiplier</code>	Совпадает с ожидаемым
4	Корректный <code>order_id</code> , время заказа в час пик (12–14 или 18–21)	<code>delivery_fee</code> увеличен на <code>peak_surcharge</code>	Совпадает с ожидаемым
5	Корректный <code>order_id</code> , день заказа — суббота или воскресенье	<code>delivery_fee</code> увеличен на <code>weekend_surcharge</code>	Совпадает с ожидаемым
6	Корректный <code>order_id</code> , первый заказ клиента	<code>discount</code> = <code>subtotal</code> × <code>p_discount_rate</code>	Совпадает с ожидаемым
7	Несуществующий <code>order_id</code>	Возврат пустого результата	Совпадает с ожидаемым

Таблица 3.3 – Классы эквивалентности для триггера обновления рейтинга ресторана

№	Класс эквивалентности	Ожидаемый результат	Фактический результат
1	INSERT отзыва с корректным <code>order_id</code> , ресторан существует	Рейтинг ресторана обновлён до среднего значения	Совпадает с ожидаемым
2	UPDATE отзыва, оценка ресторана изменена	Рейтинг ресторана пересчитан до нового среднего значения с учётом изменённой оценки	Совпадает с ожидаемым
3	INSERT отзыва, <code>restaurant_id</code> не найден через <code>orders</code>	Триггер завершается без изменений, RETURN NEW	Совпадает с ожидаемым

Таблица 3.4 – Классы эквивалентности для триггера обновления рейтинга курьера

№	Класс эквивалентности	Ожидаемый результат	Фактический результат
1	INSERT отзыва с корректным <code>order_id</code> , курьер назначен	Рейтинг курьера обновлён до среднего значения	Совпадает с ожидаемым
2	UPDATE отзыва, оценка курьера изменена	Рейтинг курьера пересчитан до нового среднего значения с учётом изменённой оценки	Совпадает с ожидаемым
3	INSERT отзыва, <code>courier_id = NULL</code> (курьер не назначен)	Триггер завершается без изменений, RETURN NEW	Совпадает с ожидаемым

3.8 Создание ролевой модели

В листингах 3.34–3.38 представлены SQL-запросы для создания ролей и назначения прав доступа в базе данных.

Листинг 3.34 – Создание ролей

```
CREATE ROLE customer_role;
CREATE ROLE restaurant_admin_role;
CREATE ROLE courier_role;
CREATE ROLE system_admin_role;
```


Листинг 3.35 – Назначение прав доступа для роли клиента

```
GRANT SELECT ON restaurants, dish_categories, dishes TO customer_role;  
GRANT SELECT, UPDATE ON customers TO customer_role;  
GRANT SELECT, INSERT, UPDATE ON customer_addresses TO customer_role;  
GRANT SELECT, INSERT ON orders TO customer_role;  
GRANT SELECT, INSERT ON order_items TO customer_role;  
GRANT SELECT ON payments TO customer_role;  
GRANT SELECT ON delivery_zones TO customer_role;  
GRANT SELECT, INSERT, UPDATE ON reviews TO customer_role;
```

Листинг 3.36 – Назначение прав доступа для роли курьера

```
GRANT SELECT, UPDATE ON couriers TO courier_role;  
GRANT SELECT, UPDATE ON orders TO courier_role;  
GRANT SELECT ON order_items TO courier_role;  
GRANT SELECT ON restaurants TO courier_role;  
GRANT SELECT ON customers TO courier_role;  
GRANT SELECT ON customer_addresses TO courier_role;  
GRANT SELECT ON delivery_zones TO courier_role;  
GRANT SELECT ON reviews TO courier_role;
```

Листинг 3.37 – Назначение прав доступа для роли администратора ресторана

```
GRANT SELECT, UPDATE ON restaurants TO restaurant_admin_role;  
GRANT SELECT, INSERT, UPDATE, DELETE ON dish_categories TO  
    restaurant_admin_role;  
GRANT SELECT, INSERT, UPDATE, DELETE ON dishes TO restaurant_admin_role;  
GRANT SELECT, UPDATE ON orders TO restaurant_admin_role;  
GRANT SELECT ON order_items TO restaurant_admin_role;  
GRANT SELECT, INSERT, UPDATE, DELETE ON delivery_zones TO  
    restaurant_admin_role;  
GRANT SELECT ON reviews TO restaurant_admin_role;
```

Листинг 3.38 – Назначение прав доступа для роли системного администратора

```
GRANT ALL PRIVILEGES ON ALL TABLES IN SCHEMA public TO system_admin_role;  
GRANT ALL PRIVILEGES ON ALL SEQUENCES IN SCHEMA public TO  
    system_admin_role;  
GRANT ALL PRIVILEGES ON ALL FUNCTIONS IN SCHEMA public TO  
    system_admin_role;
```

3.9 Реализация взаимодействия PostgreSQL с Redis

В листинге 3.39 представлен пример реализации кэширования в сервисе ресторанов.

Листинг 3.39 – Пример реализации кэширования в RestaurantService

```
@Cacheable(value = "restaurants", key = "#id")
@Transactional(readOnly = true)
public Restaurant getById(UUID id) {
    return restaurantRepository.findById(id)
        .orElseThrow(() ->
            new ResourceNotFoundException("Restaurant not found: " +
                id));
}

@Cacheable(value = "restaurants", key = "'active'")
@Transactional(readOnly = true)
public List<Restaurant> getActive() {
    return restaurantRepository.findByIsActiveTrue();
}

@CacheEvict(value = "restaurants", allEntries = true)
public Restaurant update(UUID id, UpdateRestaurantCommand command) {
    Restaurant restaurant = getById(id);
    // обновление полей...
    return restaurantRepository.save(restaurant);
}
```

Аннотация `@Cacheable` обеспечивает сохранение результата метода в Redis при первом вызове и возврат кэшированного значения при последующих вызовах с тем же ключом. Аннотация `@CacheEvict` инвалидирует кэш при изменении данных, гарантируя актуальность возвращаемых данных.

В таблице 3.5 представлены значения TTL для каждого кэша.

Таблица 3.5 – Значения TTL для кэшей Redis

Кэш	TTL	Причина
restaurants	60 минут	Данные о ресторанах изменяются редко
dishes	30 минут	Меню может обновляться в течение дня
delivery_zones	120 минут	Зоны доставки изменяются крайне редко
couriers	15 минут	Статус доступности курьера меняется часто

3.10 Описание интерфейса доступа к базе данных

В качестве интерфейса доступа к базе данных реализован REST API. Для документирования и интерактивного тестирования эндпоинтов используется Swagger UI [21] на основе спецификации OpenAPI 3.1.0 [22], позволяющий выполнять запросы к API из браузера.

На рисунке 3.1 представлена главная страница Swagger UI с перечнем доступных эндпоинтов.

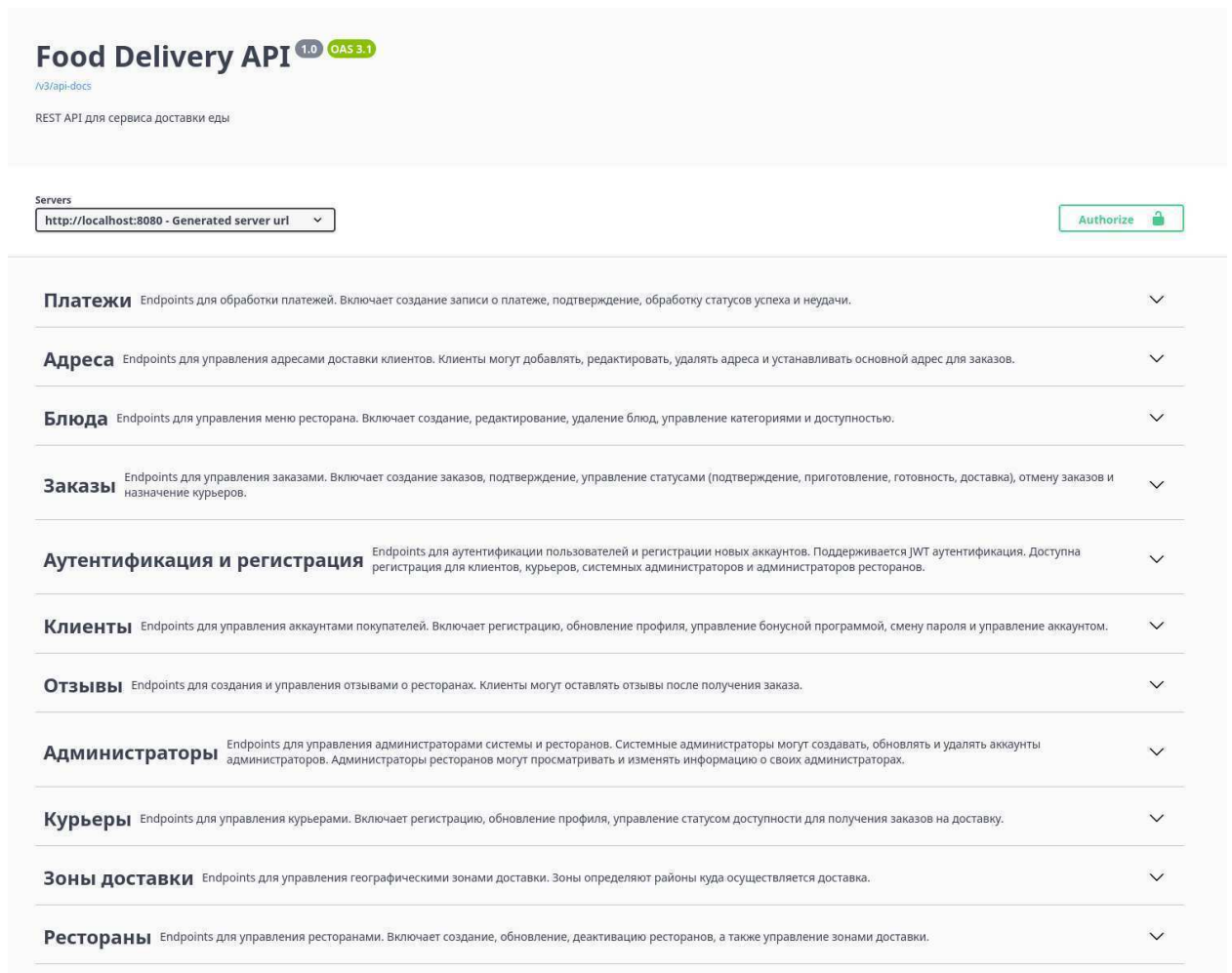


Рисунок 3.1 – Главная страница Swagger UI

На рисунке 3.2 представлен пример выполнения запроса на регистрацию клиента.

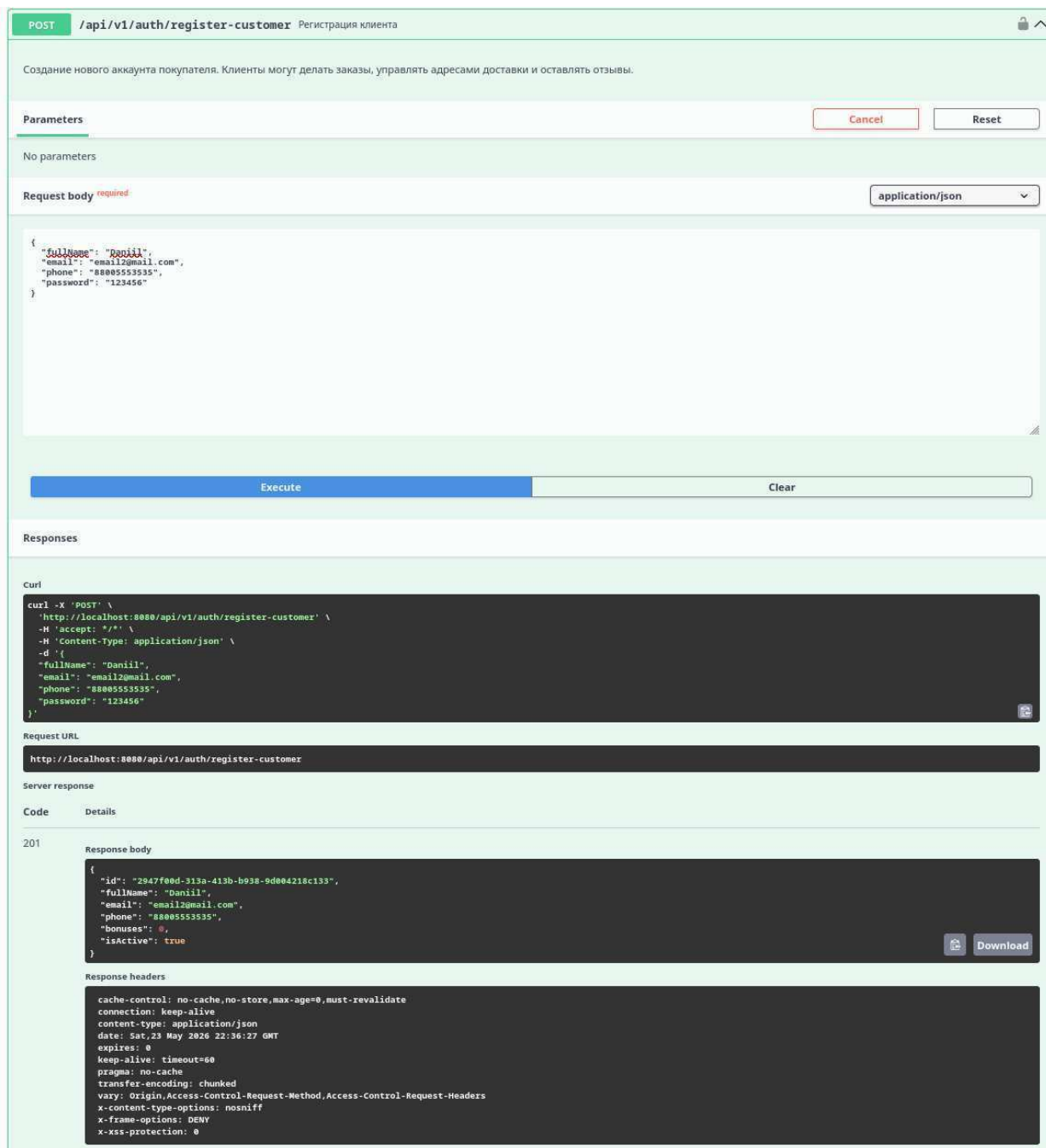


Рисунок 3.2 – Пример выполнения запроса в Swagger UI

3.11 Вывод

В технологическом разделе был обоснован выбор PostgreSQL в качестве основной СУБД, были созданы типы и таблицы базы данных и настроены ограничения их целостности. Разработаны функции и триггеры, а также приведены результаты их тестирования. Реализована ролевая модель разграничения прав доступа на уровне СУБД. На стороне приложения реализовано кэширование на базе СУБД Redis, дано описание

взаимодействия Redis с PostgreSQL, и разработан интерфейс REST API с документацией в Swagger UI.

4 Исследовательский раздел

4.1 Технические характеристики ЭВМ

Замеры выполнялись на следующей машине:

- процессор 11th Gen Intel Core i5-1135G7 4.20 ГГц [23];
- оперативная память 8 Гб;
- операционная система EndeavourOS [24] Linux 7.0.3-arch1-2 x86_64.

Указанный процессор имеет 4 физических ядра и 8 логических [23]. Исследование проводилось на ноутбуке, подключённом в сеть, при включённом режиме производительности. При выполнении замеров были запущены лишь системные процессы.

4.2 Описание исследования

Проведены два независимых исследования: влияние внешнего кэширования на время выполнения запросов и влияние индексов на скорость выполнения запросов.

4.2.1 Исследование влияния кэширования

Для исследования был выбран запрос получения списка активных ресторанов (листинг 4.1): он выполняется при каждом открытии каталога и при этом обращается к данным, которые меняются редко.

Листинг 4.1 – Запрос для исследования кэширования

```
SELECT * FROM restaurants WHERE is_active = true
```

Таблица заполнялась N записями для каждого значения из набора $N \in \{500, 1000, 1500, 2000, 2500, 3000, 3500, 4000, 4500, 5000\}$. Для каждого значения N выполнялось 50 замеров после 5 предварительных запросов. Итоговое время определялось как медиана оставшихся замеров. Время измерялось функцией `time.perf_counter()` стандартной библиотеки Python [25, 26]. Для PostgreSQL оно включает разбор запроса, построение плана, чтение данных и передачу результата клиенту. Для Redis — время операции `GET` из оперативной памяти.

4.2.2 Исследование влияния индексов

Для проведения исследования была выбрана таблица `orders`, поскольку необходимо иметь возможность быстро получать все доставленные заказы конкретного клиента. Так как запрос фильтрует строки одновременно по `customer_id` и `status`, для индексации были выбраны оба столбца.

В листинге 4.2 представлен рассматриваемый запрос, на котором проводились замеры. Значение `customer_id` подставлялось как параметр при каждом вызове. В листинге 4.3 представлено создание составного индекса в таблице `orders`.

Листинг 4.2 – Рассматриваемый запрос

```
SELECT order_id, total_amount
FROM orders
WHERE customer_id = $1 AND status = 'delivered'
```

Листинг 4.3 – Создание составного индекса

```
CREATE INDEX idx_orders_customer_status
ON orders(customer_id, status)
```

Время выполнения измерялось с использованием команды PostgreSQL `EXPLAIN (ANALYZE, TIMING)` [27]. Перед каждой серией замеров выполнялся `VACUUM ANALYZE` [28], чтобы планировщик работал с актуальной статистикой. Объём таблицы варьировался в диапазоне $N \in \{10000, 25000, 50000, 100000, 200000, 300000, 400000, 500000\}$ строк. Для каждого N проводилось 30 замеров после 5 предварительных; итоговым значением служила медиана.

4.3 Результаты исследования

4.3.1 Влияние кэширования

Результаты замеров приведены в таблице 4.1.

Таблица 4.1 – Результаты исследования влияния кэширования

Кол-во записей	PostgreSQL, мс	Redis, мс
500	1.744	0.230
1000	3.447	0.284
1500	5.252	0.397
2000	6.599	1.798
2500	8.941	2.236
3000	10.761	2.757
3500	12.244	3.139
4000	14.041	3.654
4500	16.294	4.072
5000	19.665	4.173

На рисунке 4.1 представлена диаграмма сравнения времени выполнения запросов.

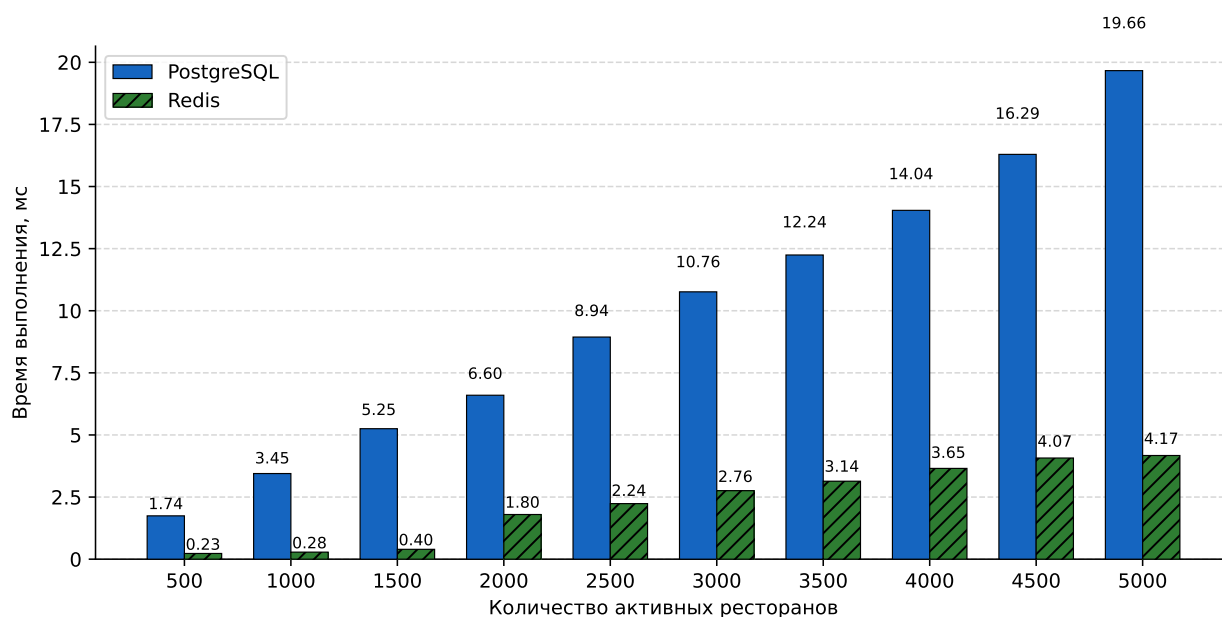


Рисунок 4.1 – Сравнение времени выполнения запроса к PostgreSQL и Redis

Для измерений PostgreSQL была выполнена линейная аппроксимация методом наименьших квадратов (формула 1):

$$y = 0.0038x - 0.57. \quad (1)$$

Для Redis аппроксимация даёт следующую зависимость (формула 2):

$$y = 0.0010x - 0.49. \quad (2)$$

Коэффициенты детерминации составили $R^2 \approx 0.991$ для PostgreSQL и $R^2 \approx 0.962$ для Redis, что говорит о линейном характере обеих зависимостей. Коэффициент при x у PostgreSQL в 3.8 раза больше, чем у Redis, то есть время PostgreSQL растёт в 3.8 раза быстрее.

4.3.2 Влияние индексов

Результаты замеров приведены в таблице 4.2.

Таблица 4.2 – Результаты исследования влияния составного индекса

Строк в таблице	С индексом, мс	Без индекса, мс
10 000	0.018	0.052
25 000	0.030	0.130
50 000	0.055	0.294
100 000	0.106	0.569
200 000	0.211	1.123
300 000	0.429	1.724
400 000	0.930	2.774
500 000	1.357	3.535

На рисунке 4.2 представлена диаграмма сравнения времени выполнения запросов.

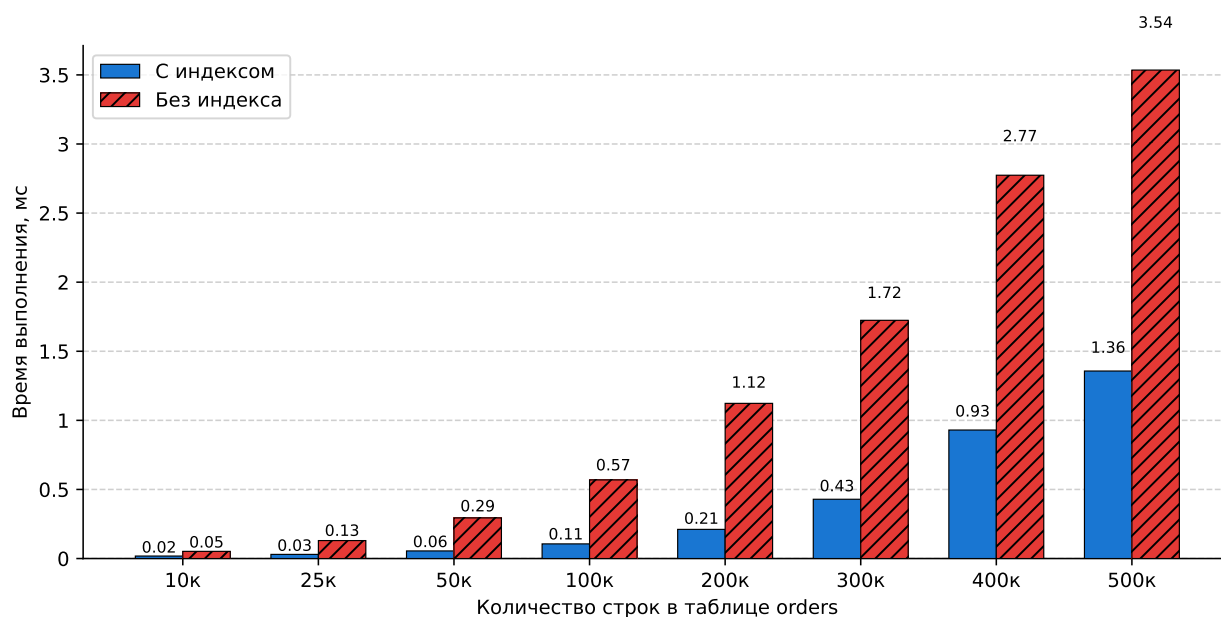


Рисунок 4.2 – Сравнение времени выполнения запроса с индексом и без

Запрос с индексом выполняется быстрее на всём исследованном диапазоне. При 10 000 строк ускорение составляет 2.9 раза, поскольку оба варианта работают с данными в памяти. В диапазоне от 25 000 до 200 000 строк ускорение стабилизируется на уровне 5.3–5.4 раза: последовательное сканирование затрагивает всё больший объём страниц, тогда как индексный поиск обращается только к нужным строкам. При дальнейшем росте таблицы ускорение снижается до 2.6 раза, так как при таком объёме оба варианта активно обращаются к дисковым страницам.

4.4 Вывод

В данном разделе приведены технические характеристики устройства, описаны исследования влияния внешнего кэширования и индексов на время выполнения запросов и представлены их результаты. Время выполнения запроса к PostgreSQL растёт линейно с увеличением объёма выборки, тогда как Redis обеспечивает значительно меньший прирост. Составной индекс ускоряет выполнение запросов в 2.6–5.4 раза в зависимости от объёма таблицы.

ЗАКЛЮЧЕНИЕ

Цель курсовой работы достигнута: разработана базы данных для системы онлайн-заказов еды, а также приложение для доступа к ней.

При этом в ходе работы были выполнены все поставленные задачи:

- проведён анализ существующих систем онлайн-заказов еды;
- определены требования и ограничения к разрабатываемой базе данных и приложению;
- спроектирована база данных и заданы ограничения целостности;
- выбраны средства реализации базы данных и приложения;
- реализованы спроектированные база данных и приложение;
- проведёно исследование влияния внешнего кэширования и индексации на время выполнения запросов к базе данных.

Разработанная база данных включает 13 таблиц, реализует ролевую модель разграничения доступа, содержит хранимую функцию расчёта итоговой стоимости заказа и триггеры автоматического обновления рейтингов. Проведённые исследования показали, что время выполнения запроса к PostgreSQL растёт линейно с увеличением объёма выборки, тогда как Redis обеспечивает значительно меньший прирост. Составной индекс ускоряет выполнение запросов в 2.6–5.4 раза в зависимости от объёма таблицы.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Доставка еды: как пандемия ускорила рост рынка [Электронный ресурс]. – URL: <https://marketing.rbc.ru/articles/14230/> (дата обращения: 08.03.2026).
2. Рынок готовой еды 2025: производство, ёмкость и перспективы развития [Электронный ресурс]. – URL: <https://expertcc.ru/articles/rynok-gotovoy-edy-2025-proizvodstvo-emkost-i-perspektivy-razvitiya> (дата обращения: 08.03.2026).
3. Сальникова К. В. Исследование российского рынка доставки готовой еды // Социально-экономическое управление: теория и практика. 2025. Т. 21, № 3. С. 30–45.
4. Мартынов К. В. Принципы ценообразования в сфере доставки готовых блюд: анализ факторов влияния // Путеводитель предпринимателя. 2025. Т. 18, № 2. С. 74–82.
5. Управление рейтингом ресторана [Электронный ресурс]. – URL: <https://yandex.com/support/eda-vendor-ru/ru/rating/rating-up> (дата обращения: 27.05.2026).
6. Яндекс Еда [Электронный ресурс]. – URL: <https://eda.yandex.ru> (дата обращения: 17.03.2026).
7. Деливери [Электронный ресурс]. – URL: <https://market-delivery.yandex.ru> (дата обращения: 17.03.2026).
8. Купер [Электронный ресурс]. – URL: <https://kuper.ru> (дата обращения: 20.04.2026).
9. О.Е. Аврунев В.М. Стасышин. Модели баз данных: учебное пособие. Новосибирск: Изд-во НГТУ, 2018. 124 с.
10. Дейт К. Дж. Введение в системы баз данных. 8-е издание. Москва: Издательский дом «Вильямс», 2005. 1328 с.

11. Б. А. Новиков Е. А. Горшкова Н. Г. Графеева. Основы технологий баз данных: учебное пособие / под ред. Е. В. Рогова. 2-е изд. Москва: ДМК Пресс, 2020. 582 с.
12. PostgreSQL [Электронный ресурс]. – URL: <https://www.postgresql.org/> (дата обращения: 17.05.2026).
13. MySQL [Электронный ресурс]. – URL: <https://www.mysql.com/> (дата обращения: 17.05.2026).
14. Microsoft SQL Server [Электронный ресурс]. – URL: <https://www.microsoft.com/ru-ru/sql-server> (дата обращения: 17.05.2026).
15. Oracle Database [Электронный ресурс]. – URL: <https://www.oracle.com/database/> (дата обращения: 17.05.2026).
16. Redis [Электронный ресурс]. – URL: <https://redis.io/> (дата обращения: 24.05.2026).
17. Java [Электронный ресурс]. – URL: <https://www.java.com/ru/> (дата обращения: 17.05.2026).
18. Spring Boot [Электронный ресурс]. – URL: <https://spring.io/projects/spring-boot> (дата обращения: 17.05.2026).
19. Spring Data JPA [Электронный ресурс]. – URL: <https://spring.io/projects/spring-data-jpa> (дата обращения: 17.05.2026).
20. Spring Data Redis [Электронный ресурс]. – URL: <https://spring.io/projects/spring-data-redis> (дата обращения: 24.05.2026).
21. Swagger UI [Электронный ресурс]. – URL: <https://swagger.io/tools/swagger-ui/> (дата обращения: 24.05.2026).

22. OpenAPI Specification [Электронный ресурс]. – URL: <https://spec.openapis.org/oas/v3.1.0.html> (дата обращения: 24.05.2026).
23. Intel Core i5-1135G7 Processor [Электронный ресурс]. – URL: <https://www.intel.com/content/www/us/en/products/sku/208922/intel-core-i51135g7-processor8m-cache-up-to-4-20-ghz-with-ipu/specifications.html> (дата обращения: 27.05.2026).
24. EndeavourOS [Электронный ресурс]. – URL: <https://endeavouros.com/> (дата обращения: 27.05.2026).
25. Python [Электронный ресурс]. – URL: <https://www.python.org/> (дата обращения: 28.05.2026).
26. Time access and conversions [Электронный ресурс]. – URL: https://docs.python.org/3/library/time.html#time.perf_counter (дата обращения: 28.05.2026).
27. EXPLAIN [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/current/sql-explain.html> (дата обращения: 28.05.2026).
28. VACUUM [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/current/sql-vacuum.html> (дата обращения: 28.05.2026).

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе состоит из N слайдов.

ПРИЛОЖЕНИЕ Б

Листинг Б.1 – Функция подсчёта итоговой суммы заказа. Часть 1

```
CREATE OR REPLACE FUNCTION calculate_order_total(  
    p_order_id UUID,  
    p_discount_rate NUMERIC DEFAULT 0.2  
)  
    RETURNS TABLE  
    (  
        subtotal      NUMERIC ,  
        delivery_fee  NUMERIC ,  
        discount      NUMERIC ,  
        total_amount  NUMERIC  
    )  
    LANGUAGE plpgsql  
AS  
$$  
DECLARE  
    v_order          orders%ROWTYPE;  
    v_restaurant     restaurants%ROWTYPE;  
    v_zone           delivery_zones%ROWTYPE;  
    v_subtotal       NUMERIC;  
    v_delivery_fee   NUMERIC;  
    v_additional_fee NUMERIC;  
    v_discount       NUMERIC := 0;  
    v_distance       NUMERIC;  
    v_hour           INTEGER;  
    v_day_of_week    TEXT;  
    v_orders_count   INTEGER;  
    v_postal_code    TEXT;  
BEGIN  
    SELECT *  
    INTO v_order  
    FROM orders  
    WHERE order_id = p_order_id;  
  
    SELECT *  
    INTO v_restaurant  
    FROM restaurants  
    WHERE restaurant_id = v_order.restaurant_id;  
  
    SELECT ca.postal_code  
    INTO v_postal_code  
    FROM customer_addresses ca  
    WHERE ca.customer_id = v_order.customer_id AND ca.is_default = TRUE  
    LIMIT 1;
```


Листинг Б.2 – Функция подсчёта итоговой суммы заказа. Часть 2

```

SELECT *
INTO v_zone
FROM delivery_zones
WHERE restaurant_id = v_order.restaurant_id
AND postal_code = v_postal_code
LIMIT 1;
SELECT COALESCE(SUM(unit_price * quantity), 0)
INTO v_subtotal
FROM order_items
WHERE order_id = p_order_id;
IF v_subtotal >= v_zone.min_order_amount THEN
    v_delivery_fee := 0;
ELSE
    v_distance := (
        2 * 6371 * ASIN(SQRT(
            POWER(SIN(RADIANS((v_order.delivery_latitude -
                v_restaurant.latitude) / 2)), 2) +
            COS(RADIANS(v_restaurant.latitude)) *
            COS(RADIANS(v_order.delivery_latitude)) *
            POWER(SIN(RADIANS((v_order.delivery_longitude -
                v_restaurant.longitude) / 2)), 2)
        ))
    );

    v_delivery_fee := v_zone.delivery_fee;
    IF v_distance > v_zone.near_threshold THEN
        v_additional_fee := (v_distance - v_zone.near_threshold) *
            v_zone.fee_per_km;
        IF v_distance > v_zone.far_threshold THEN
            v_additional_fee := v_additional_fee *
                v_zone.far_zone_multiplier;
        END IF;
        v_delivery_fee := v_delivery_fee + v_additional_fee;
    END IF;
    v_hour := EXTRACT(HOUR FROM v_order.created_at);
    IF (v_hour >= 12 AND v_hour < 14) OR (v_hour >= 18 AND v_hour <
        21) THEN
        v_delivery_fee := v_delivery_fee + v_zone.peak_surcharge;
    END IF;
    v_day_of_week := TO_CHAR(v_order.created_at, 'Day');
    IF TRIM(v_day_of_week) = 'Saturday' OR TRIM(v_day_of_week) =
        'Sunday' THEN
        v_delivery_fee := v_delivery_fee + v_zone.weekend_surcharge;
    END IF;
END IF;

```

Листинг Б.3 – Функция подсчёта итоговой суммы заказа. Часть 3

```

SELECT COUNT(*)
INTO v_orders_count
FROM orders
WHERE customer_id = v_order.customer_id
      AND order_id != p_order_id;
v_discount := 0;
IF v_orders_count = 0 THEN
    v_discount := v_subtotal * p_discount_rate;
END IF;
RETURN QUERY SELECT ROUND(v_subtotal, 2),
                      ROUND(v_delivery_fee, 2),
                      ROUND(v_discount, 2),
                      ROUND(v_subtotal + v_delivery_fee - v_discount, 2);
END;
$$;

```

Листинг Б.4 – Триггер для автоматического обновления рейтинга ресторана

```

CREATE OR REPLACE FUNCTION update_restaurant_rating()
RETURNS TRIGGER AS $$
DECLARE
    v_restaurant_id  UUID;
    v_new_rating      NUMERIC;
BEGIN
    SELECT o.restaurant_id INTO v_restaurant_id
    FROM orders o
    WHERE o.order_id = NEW.order_id;
    IF v_restaurant_id IS NULL THEN
        RETURN NEW;
    END IF;
    SELECT ROUND(AVG(r.restaurant_rating)::NUMERIC, 2) INTO v_new_rating
    FROM reviews r
    JOIN orders o ON o.order_id = r.order_id
    WHERE o.restaurant_id = v_restaurant_id;
    UPDATE restaurants
    SET rating      = v_new_rating,
        updated_at = NOW()
    WHERE restaurant_id = v_restaurant_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_restaurant_rating
AFTER INSERT OR UPDATE ON reviews
FOR EACH ROW
EXECUTE FUNCTION update_restaurant_rating();

```

Листинг Б.5 – Триггер для автоматического обновления рейтинга курьера

```
CREATE OR REPLACE FUNCTION update_courier_rating()
RETURNS TRIGGER AS $$
DECLARE
    v_courier_id  UUID;
    v_new_rating  NUMERIC;
BEGIN
    SELECT o.courier_id INTO v_courier_id
    FROM orders o
    WHERE o.order_id = NEW.order_id;
    IF v_courier_id IS NULL THEN
        RETURN NEW;
    END IF;
    SELECT ROUND(AVG(r.courier_rating)::NUMERIC, 2) INTO v_new_rating
    FROM reviews r
    JOIN orders o ON o.order_id = r.order_id
    WHERE o.courier_id = v_courier_id;
    UPDATE couriers
    SET rating      = v_new_rating,
        updated_at = NOW()
    WHERE courier_id = v_courier_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_update_courier_rating
AFTER INSERT OR UPDATE ON reviews
FOR EACH ROW
EXECUTE FUNCTION update_courier_rating();
```